Part 1 – Requirements Engineering

Collaborative Document Editor with AI Writing Assistant

Alexander Danchev, Anel Murat, Yintong Wang

February 2026

Contents

1 Stakeholder Analysis **4**
1.1 Corporate Users (B2B) . . . 4
1.2 Document Users (B2C) . . . 5
1.3 Workspace Admins . . . 5
1.4 Maintenance Team . . . 5
1.5 Project Team . . . 6
1.6 Department Managers . . . 6
1.7 Business Team . . . 7
1.8 Investors . . . 7
1.9 Regulators . . . 7
1.10 LLM & Cloud Provider . . . 7
1.11 Stakeholder Tensions and Conflicts . . . 8

2 Functional Requirements **9**
2.1 Real-Time Collaboration . . . 9
2.1.1 FR-RTC-1 – Simultaneous Editing (MUST) . . . 9
2.1.2 FR-RTC-2 – Presence Awareness (SHOULD) . . . 9
2.1.3 FR-RTC-3 – Conflict Resolution (MUST) . . . 10
2.1.4 FR-RTC-4 – Change Indicators (COULD) . . . 10
2.1.5 FR-RTC-5 – Offline Editing and Reconnection (SHOULD) . . . 10
2.2 AI Writing Assistant . . . 10
2.2.1 FR-AI-1 – AI Invocation (MUST) . . . 11
2.2.2 FR-AI-2 – Text Enhancement / Rewriting (MUST) . . . 11
2.2.3 FR-AI-3 – Summarization (SHOULD) . . . 11
2.2.4 FR-AI-4 – Translation (SHOULD) . . . 12
2.2.5 FR-AI-5 – Suggestion Review (MUST) . . . 12
2.2.6 FR-AI-6 – AI Interaction Logging (SHOULD) . . . 12
2.2.7 FR-AI-7 – AI Feature Policy and Quota Management (MUST) . . . 13
2.3 Document Management . . . 13
2.3.1 FR-DOC-1 – Document Creation (MUST) . . . 13
2.3.2 FR-DOC-2 – Version History (MUST) . . . 14
2.3.3 FR-DOC-3 – Sharing and Permissions (MUST) . . . 14
2.3.4 FR-DOC-4 – Exporting Documents (SHOULD) . . . 14
2.3.5 FR-DOC-5 – Commenting (MUST) . . . 14

2.3.6 FR-DOC-6 – Document Organization (SHOULD) . . . 15
2.4 User Management . . . 15
2.4.1 FR-USR-1 – Authentication (MUST) . . . 15
2.4.2 FR-USR-2 – Role-Based Authorization (MUST) . . . 16
2.4.3 FR-USR-3 – Session Management (SHOULD) . . . 16
2.4.4 FR-USR-4 – User Profile Management (COULD) . . . 16
2.4.5 FR-USR-5 – Access Request Notifications (SHOULD) . . . 17

3 Non-Functional Requirements **18**
3.1 System Performance . . . 18
3.1.1 NFR-PERF-1 – Keystroke Propagation Latency (MUST) . . . 18
3.1.2 NFR-PERF-2 – AI Response Initiation (MUST) . . . 18
3.1.3 NFR-PERF-3 – Document Load Time (MUST) . . . 19
3.2 Scalability . . . 19
3.2.1 NFR-SCL-1 – Concurrent Editors per Document (MUST) . . . 20
3.2.2 NFR-SCL-2 – Concurrent Documents (MUST) . . . 20
3.2.3 NFR-SCL-3 – Horizontal Scalability (SHOULD) . . . 20
3.3 Availability . . . 21
3.3.1 NFR-AVL-1 – Service Availability (MUST) . . . 21
3.3.2 NFR-AVL-2 – Editing Session Resilience (MUST) . . . 21
3.3.3 NFR-AVL-3 – AI Service Failure Handling (SHOULD) . . . 21
3.3.4 NFR-AVL-4 – Disaster Recovery (MUST) . . . 22
3.4 Security and Privacy . . . 22
3.4.1 NFR-SEC-1 – Data Encryption in Transit (MUST) . . . 22
3.4.2 NFR-SEC-2 – Data Encryption at Rest (MUST) . . . 22
3.4.3 NFR-SEC-3 – AI Processing Privacy (SHOULD) . . . 23
3.4.4 NFR-SEC-4 – AI Interaction Logging (SHOULD) . . . 23
3.4.5 NFR-SEC-5 – Data Retention and Right to Erasure (SHOULD) . . . 23
3.4.6 NFR-SEC-6 – Prompt Injection Defense (SHOULD) . . . 24
3.5 Usability . . . 24
3.5.1 NFR-USB-1 – Large Document Handling (SHOULD) . . . 24
3.5.2 NFR-USB-2 – Collaboration UI at Scale (SHOULD) . . . 25
3.5.3 NFR-USB-3 – Accessibility (SHOULD) . . . 25
3.5.4 NFR-USB-4 – Internationalization and RTL Support (SHOULD) . . . 26
3.6 Data Consistency . . . 27
3.6.1 NFR-CON-1 – Document State Convergence (MUST) . . . 27

4 User Stories and Scenarios **27**
4.1 Collaboration Scenarios . . . 27
4.1.1 US-1: Simultaneous Paragraph Editing . . . 27
4.1.2 US-2: Offline Editing and Reconnection . . . 28
4.1.3 US-3: Reverting to a Previous Version During Active Collaboration . . . 28
4.1.4 US-4: Presence Awareness in a Crowded Document . . . 29
4.2 AI Assistant Workflows . . . 29
4.2.1 US-5: Summarizing a Long Section . . . 29
4.2.2 US-6: Translating a Section into Another Language . . . 30
4.2.3 US-7: Partially Accepting an AI Rewrite . . . 30
4.2.4 US-8: AI Invocation During Active Collaboration . . . 31

4.2.5 US-9: Hitting the AI Usage Quota . . . 31
4.2.6 US-10: Restructuring Notes into a Document Outline . . . 32
4.3 Document Lifecycle . . . 33
4.3.1 US-11: Sharing a Document with Read-Only Access . . . 33
4.3.2 US-12: Exporting with AI Change Tracking . . . 33
4.3.3 US-13: Reviewing AI Interaction History . . . 34
4.4 Access Control and Roles . . . 35
4.4.1 US-14: Commenter Attempts to Use the AI Assistant . . . 35
4.4.2 US-15: Workspace Admin Configures AI Policies per Role . . . 36
4.4.3 US-16: Viewer Attempts to Edit . . . 36

5 Requirements Traceability **37**
5.1 Requirement Validation Mapping . . . 37
5.2 User Story Traceability Matrix . . . 37
5.3 Requirement Coverage Matrix . . . 39
5.4 Non-Functional Requirement Coverage . . . 40
5.5 Traceability Observations . . . 41

1 Stakeholder Analysis

We identified ten stakeholder groups, and classified each stakeholder along five axes:

1. **Internal–External** from company perspective
2. **Primary–Secondary** – whether they directly use the product or are indirectly affected
3. **Technical–Business** orientation
4. **Power** (High / Medium / Low)
5. **Interest** (High / Medium / Low)

The table below gives a quick overview before we go into detail on each group.

Stakeholder	Int/Ext	Pri/Sec	Power	Interest	Tech/Bus
Corporate Users (B2B)	Ext	Primary	High	High	Business
Document Users (B2C)	Ext	Primary	Low	High	Business
Workspace Admins	Ext	Primary	High	High	Both
Maintenance Team	Int	Secondary	Medium	High	Technical
Project Team	Int	Secondary	Medium	High	Technical
Dept. Managers	Int	Secondary	High	Medium	Business
Business Team	Int	Secondary	Medium	High	Business
Investors	Ext	Secondary	High	Medium	Business
Regulators	Ext	Secondary	High	Low–Med	Business
LLM & Cloud Provider	Ext	Secondary	Med–High	Medium	Technical

Table 1: Stakeholder classification summary

1.1 Corporate Users (B2B)

Classification: External, Primary, Business, High Power, High Interest

These are the enterprise customers – companies paying for the product. They carry heavy weight, because they're likely the main revenue source and their needs will probably shape the product roadmap.

Goals. They want support for team-based workflows where multiple people edit at once, productivity gains from AI features like summarization and rewriting, translation, and smooth integration with whatever tools they already use (e.g. Slack, Google Drive, Outlook, Teams, etc.). Most importantly, they need reliability – a system that goes down during a contract deadline is a serious problem.

Concerns. Privacy is the big one. Sensitive corporate documents being sent to third-party LLM APIs is a hard sell for legal and IT departments. Beyond that: system downtime during critical moments, and making sure access control is granular enough that a junior hire can't stumble into a board-level document. There's also the question of compliance – many enterprises need SOC 2 or even HIPAA, depending on industry.

Influence on Requirements. This stakeholder group pushes us toward enterprise-grade auth including SSO via SAML/OAuth (FR-USR-1), strong encryption both at rest via AES-256 (NFR-SEC-2) and in transit via TLS 1.2+ (NFR-SEC-1), a disaster recovery plan with concrete RTO/RPO targets (NFR-AVL-4), document version control with audit trails (FR-DOC-2),

workspace management with organization-level settings (FR-USR-2), AI feature policy controls (FR-AI-7), and well-documented APIs so their engineering teams can integrate our tool into existing pipelines.

1.2 Document Users (B2C)

Classification: External, Primary, Business, Low Power, High Interest

Individual users – students, instructors, freelancers, personal use. They don't have as much power as enterprise clients (i.e. they're not paying thousands per seat), but there are a lot of them and their experience shapes word-of-mouth and reviews.

Goals. An intuitive interface they can pick up without a tutorial. AI assistance that actually helps rather than getting in the way. Access from any device – laptop, tablet, phone – without a degraded experience. And fast performance: nobody wants to wait three seconds for the page to load.

Concerns. They worry about privacy, especially since they may not fully understand what happens when their text gets sent to an AI service. Losing unsaved changes is a common fear (and a common experience with other tools). They also worry about AI suggestions being irrelevant or annoying, and about the UI becoming confusing when multiple people are editing.

Influence on Requirements. This drives us toward an intuitive UI with progressive disclosure, robust version history and autosave (FR-DOC-2), offline editing with auto-sync on reconnection (FR-RTC-5), fast document load times (NFR-PERF-3), and a non-intrusive AI suggestion flow – inline diffs with clear accept / reject, not a popup that blocks your work (FR-AI-5).

1.3 Workspace Admins

Classification: External, Primary, Business/Technical, High Power, High Interest

These are the people managing user access and enforcing company policies within the platform. In practice they're often IT managers at client companies. They have high power, because a bad admin experience can prevent a deal.

Goals. Managing users (invite, suspend, assign roles), monitoring activity and usage patterns, and enforcing organizational policies – things like restricting AI features for certain roles, or mandating document retention periods.

Concerns. Unauthorized access due to misconfigured permissions, insufficient audit data for incident investigation or compliance audits, and not being able to control which AI features are available to whom.

Influence on Requirements. Role-based access control with configurable roles (FR-USR-2), admin-configurable AI feature policies including per-role toggles and per-organization quotas (FR-AI-7), an admin dashboard showing usage analytics (active users, document activity, AI invocation counts), policy enforcement features, user management tools with bulk operations and SSO provisioning, and an exportable audit log (FR-AI-6).

1.4 Maintenance Team

Classification: Internal, Secondary, Technical, Medium Power, High Interest

Customer support, IT operations, SREs – the people who keep the system running *after* launch. Unlike the Project Team (who build the system during development), the Maintenance Team's concern is production health: uptime, incident response, and user support. Their work is inherently *post-deployment*, focused on observability and operational stability rather than feature development.

Goals. Maintain uptime to meet our 99.9% target (NFR-AVL-1), diagnose and fix issues fast (especially real-time sync failures and AI degradation), and have enough context to provide useful support when users file tickets.

Concerns. Without good observability tools, finding the root cause of an issue in a distributed system (real-time sync + AI pipeline + document storage) is painful. They also worry about cascading failures – if the AI service goes down, it shouldn't take the editor down with it (NFR-AVL-3).

Influence on Requirements. This means we need an operational dashboard with real-time metrics (latency, error rates, active connections, AI health), automated alerts when things go wrong (latency > 1s, error rate > 1%), structured logging with distributed tracing, and documented runbooks for common incidents.

1.5 Project Team

Classification: Internal, Secondary, Technical, Medium Power, High Interest

That's developers, testers, DevOps, UI designers. Where the Maintenance Team cares about the system in production, the Project Team's focus is on the *development*: architecture, code quality, testability, and shipping velocity. Our interest is in how the system is structured so that it can be built, tested, and iterated on efficiently.

Goals. Ship features quickly with clean module boundaries. Keep code quality high through testing and code review. Avoid stepping on each other's toes when working in parallel.

Concerns. AI integration is difficult: prompts are hard to test, outputs are non-deterministic, and model APIs can change. Testing real-time collaboration (concurrent edits, conflict resolution, reconnection) is also hard to automate. And with multiple people working on overlapping areas, merge conflicts and integration issues are a real risk.

Influence on Requirements. Modular architecture with clear service boundaries, a solid testing strategy (unit, integration, E2E), AI prompts stored as configurable templates rather than hardcoded strings, and comprehensive API contracts (ideally OpenAPI specs) so frontend and backend can develop in parallel.

1.6 Department Managers

Classification: Internal, Secondary, Business, High Power, Medium Interest

They control budget and schedule. Their main concern is whether we deliver on time and within budget.

Goals. On-time delivery, minimal operational costs (especially LLM API spend, which scales with usage), and efficient resource allocation across modules.

Concerns. Schedule slips from underestimating the complexity of real-time sync or AI integration. Budget overruns if LLM API costs spike unexpectedly – for an AI-powered SaaS product, LLM cost-per-user is the single largest variable expense, and without controls it can erode margins before the product reaches scale. And the difficulty of staffing cross-cutting features – the AI assistant alone touches frontend, backend, and the sync layer, creating a tension between shipping the technically complex real-time collaboration engine (high risk, foundational) and the AI assistant (the marketable differentiator that drives adoption).

Influence on Requirements. A clearly scoped MVP, measurable performance targets (so we know if we're on track), a priority ranking of features (real-time editing first, AI assistant second, advanced admin tools later), and cost controls like LLM caching, per-user quotas, and tiered model selection (FR-AI-7).

1.7 Business Team

Classification: Internal, Secondary, Business, Medium Power, High Interest

Business analytics, sales, marketing. They care about growth and differentiation.

Goals. Attract new customers (the AI assistant is the key differentiator), retain existing ones by understanding what keeps them engaged, and have data to inform product and marketing decisions.

Concerns. If we don't have usage data, they're flying blind. High churn from a poor onboarding experience or from not being different enough from Google Docs. And for enterprise sales, they need to be able to show the ROI of AI features during demos.

Influence on Requirements. Product analytics tracking (feature adoption, AI invocation frequency, session metrics), a business dashboard with engagement and retention data, A/B testing infrastructure for experimenting with onboarding flows, and feature-level usage reports for sales enablement. AI interaction logging (FR-AI-6) feeds directly into these analytics needs.

1.8 Investors

Classification: External, Secondary, Business, High Power, Medium Interest

They want to see graphs go up and right – user growth, revenue, profit, efficient capital use.

Goals. User base growth that proves product-market fit, revenue from free-to-paid conversion, and efficient capital use – especially since LLM API costs are the main variable expense. Specific to this product, investors are watching whether AI features drive *retention* (users come back because the AI is genuinely useful) rather than just novelty adoption (users try AI once and never again). They also want evidence that the cost-per-AI-invocation decreases with scale through caching and optimization, not increases linearly.

Concerns. Unpredictable API costs eroding margins at scale, slow adoption in a crowded market (Google Docs, Notion, etc.), and the risk that incumbents just bolt on their own AI features – making our differentiator temporary.

Influence on Requirements. Cost-efficient architecture including caching, tiered model selection, and usage quotas (FR-AI-7), investor-visible metrics (growth, revenue, cost-per-user, AI utilization), and infrastructure that scales horizontally without a rewrite (NFR-SCL-3).

1.9 Regulators

Classification: External, Secondary, Business, High Power, Low–Medium Interest

Government agencies, legal teams, compliance officers.

Goals. Compliance with data protection laws (GDPR, CCPA), protection of user privacy when content hits third-party AI services, and ethical AI practices.

Concerns. Data breaches and improper PII handling. The risk that document content gets used to train LLM models without user consent – a grey area that regulators are increasingly scrutinizing. Non-compliance with GDPR's right to erasure, data portability, and purpose limitation.

Influence on Requirements. AI transparency (disclosing when content is AI-generated), data processing agreements with LLM providers that prohibit training on user data, compliance with the EU AI Act, GDPR, and CCPA including data residency options and erasure support (NFR-SEC-5), and a privacy-by-design approach – minimal data in AI requests (NFR-SEC-3), no PII, configurable retention.

1.10 LLM & Cloud Provider

Classification: External, Secondary, Technical, Medium–High Power, Medium Interest

The company providing the AI model and/or cloud hosting. They're a dependency – if their service degrades, our AI features degrade. That gives them significant power.

Note: We group the LLM provider (e.g., OpenAI, Anthropic) and the cloud infrastructure provider (e.g., AWS, GCP) into a single stakeholder category because their influence on our requirements overlaps significantly: both are external technical dependencies whose outages degrade our system. In practice, these may be separate vendors with distinct concerns. We treat them jointly for brevity, but our architecture accounts for this distinction through the LLM abstraction layer (NFR-AVL-3) and independent scaling of cloud infrastructure (NFR-SCL-3).

Goals. Provide reliable AI services and keep our usage high (good for their revenue).

Concerns. Misuse of their APIs (prompt injection via user documents, rate limit abuse), outages that reflect badly on them, and the possibility that we switch to a competitor.

Influence on Requirements. We need an abstraction layer over the LLM API so we're not locked into one provider, fallback mechanisms for graceful degradation (NFR-AVL-3), response caching for common operations (repeated translations, etc.), rate limiting per user and per organization (FR-AI-7), and input sanitization against prompt injection (NFR-SEC-6).

1.11 Stakeholder Tensions and Conflicts

Not all stakeholder interests align. Identifying these tensions is critical because they force design trade-offs that shape the architecture.

Tension	Conflicting Interests	Resolution Strategy
Privacy vs. Analytics Corporate Users / Regulators vs. Business Team	Minimal data collection vs. rich AI usage analytics for sales and product decisions.	Log only metadata (operation type, text lengths, outcome). Aggregated dashboards for business; individual logs restricted (FR-AI-6, NFR-SEC-4).
AI Power vs. Cost Control B2C Users / Business Team vs. Investors / Managers	Unrestricted AI features for adoption vs. controlling LLM API costs that scale with usage.	Tiered quotas, response caching, lighter models for simple tasks. Configurable per-org budgets; enterprise clients pay for more, free-tier gets reasonable limits (FR-AI-7).
Observability vs. Privacy Maintenance Team vs. Regulators / Corporate Users	Extensive logging for debugging vs. minimal data retention with no sensitive content in logs.	Structured metadata-only logging; no document content. 30-day retention with auto-purge. Anonymized session IDs for tracing (NFR-SEC-4).
Free Experience vs. Conversion B2C Users vs. Investors	Full free AI access vs. free-to-paid conversion for revenue viability.	Generous free tier demonstrating AI value, with quota limits nudging power users toward paid plans. Useful enough for word-of-mouth, with room for upsell.
Enterprise Lockdown vs. User Autonomy Workspace Admins vs. Document Users	Role-based AI restrictions and retention policies vs. frictionless full-feature access.	Admin policies override within workspaces (FR-AI-7, FR-USR-2); personal documents retain full autonomy. Clear messaging on policy-restricted features.

Table 2: Key stakeholder tensions and resolution strategies

2 Functional Requirements

We organized functional requirements into four areas matching the assignment spec. Each requirement has a unique ID, a MoSCoW priority, a trigger, expected behavior, and acceptance criteria. We tried to keep the criteria actually testable – numbers where we could come up with reasonable ones, concrete conditions otherwise.

2.1 Real-Time Collaboration

Capability Statement: *The system shall enable multiple users to simultaneously edit a shared document with real-time visibility of each other's changes, automatic conflict resolution, and resilience to network disruptions.*

2.1.1 FR-RTC-1 – Simultaneous Editing (MUST)

Multiple users must be able to edit the same document concurrently with changes visible to everyone.

Trigger: Two or more users open the same document in editing mode.

Behavior: Edits from each user are synced and reflected across all active sessions through the real-time sync service.

Acceptance Criteria:

- No edits are lost when operations overlap
- Changes show up for other users within 500ms under normal network conditions
- Supports up to 50 simultaneous editors per document
- All sessions converge to the same document state within 2 seconds of the last conflicting operation

2.1.2 FR-RTC-2 – Presence Awareness (SHOULD)

The system should show who else is in the document and where their cursor is. This is what makes collaboration feel *live* rather than just concurrent.

Trigger: A user opens a document.

Behavior: A list of active collaborators appears in the UI. Each person's cursor position and text selection are visible in the editor, color-coded per user.

Acceptance Criteria:

- Each active user gets a unique color and display name
- Cursor positions update within 500ms
- Presence indicators disappear within 5 seconds of a user disconnecting
- At least 12 distinct cursor colors in the palette

2.1.3 FR-RTC-3 – Conflict Resolution (MUST)

When two people edit the same paragraph at the same time, the system can't just pick one and throw the other away. We need a deterministic merge strategy.

Trigger: Two or more users modify overlapping text within the same sync window.

Behavior: An OT (Operational Transformation) or CRDT algorithm merges the changes automatically.

Acceptance Criteria:

- No document corruption from concurrent overlapping edits
- Nobody's text disappears as a side effect of conflict resolution
- All clients converge to the same state within 2 seconds of the last conflicting operation
- The whole process is invisible to users – no manual "resolve conflict" dialogs

2.1.4 FR-RTC-4 – Change Indicators (COULD)

A nice-to-have: briefly highlighting text that was just changed by someone else, so you can see what's happening without reading a diff.

Trigger: A collaborator modifies text.

Behavior: The modified region is temporarily highlighted in that collaborator's color.

Acceptance Criteria:

- Highlight stays visible for at least 3 seconds
- Color matches the collaborator's presence indicator
- Doesn't interfere with editing or conflict resolution

2.1.5 FR-RTC-5 – Offline Editing and Reconnection (SHOULD)

People lose internet. The app shouldn't just freeze when that happens – they should be able to keep typing and have everything sync up when they're back online.

Trigger: A user's network connection drops while editing.

Behavior: Edits are cached locally. On reconnection, they're synced with the server using the same conflict resolution algorithm as FR-RTC-3.

Acceptance Criteria:

- Users can keep editing without interruption while offline
- Cached edits sync within 5 seconds of reconnection
- No edits are lost during the offline window
- A visual indicator shows when you're offline and when sync is restored

2.2 AI Writing Assistant

Capability Statement: *The system shall provide an integrated AI writing assistant that enables users to enhance, summarize, translate, and reformat selected text through a non-intrusive interface, with clear preview-before-commit semantics, configurable access controls, and usage-aware quota management.*

2.2.1 FR-AI-1 – AI Invocation (MUST)

Trigger: User selects text and clicks the AI button or hits a keyboard shortcut.
Behavior: An action menu appears with the available AI operations.
Acceptance Criteria:

- Menu appears within 300ms
- Options include at least: Rewrite, Summarize, Translate, Reformat
- If the user's role doesn't have AI permissions (see FR-USR-2), the button is either hidden or shows a tooltip explaining why
- If the user has exceeded their AI quota (see FR-AI-7), the menu shows a "quota exceeded" message with the reset time

2.2.2 FR-AI-2 – Text Enhancement / Rewriting (MUST)

The user selects some text and asks the AI to rewrite it – make it more formal, more concise, whatever the prompt dictates.

Trigger: User selects text and picks "Rewrite."

Behavior: The selected text (+ some surrounding context) is sent to the AI. A rewritten version comes back and is shown as a preview.

Acceptance Criteria:

- Response starts streaming within 3 seconds, finishes within 6 seconds for selections up to 500 words
- For selections exceeding 2,000 words, the system displays a warning and processes in chunks with a progress indicator; total time may exceed 6 seconds proportionally
- Original text stays untouched until the user explicitly accepts
- Preview appears as an inline diff or in a side panel – user can compare before committing
- Accepting creates a version history entry so the change is traceable

2.2.3 FR-AI-3 – Summarization (SHOULD)

Trigger: User selects text and picks "Summarize."
Behavior: The AI generates a condensed version of the selection.
Acceptance Criteria:

- Summary is at most 30% the length of the original
- User can insert it at the cursor or discard it
- Generated within 5 seconds for selections under 2,000 words
- For selections exceeding 2,000 words, the system processes in chunks and shows a progress indicator

2.2.4 FR-AI-4 – Translation (SHOULD)

Trigger: User selects text and picks "Translate."

Behavior: A language picker appears. The AI translates the text into the chosen language and shows a preview.

Acceptance Criteria:

- At least 20 languages supported (including English, Russian, French, German, Spanish, Polish, Arabic, Chinese, Japanese)
- Preview shown before insertion
- If a language isn't supported, the system says so and shows what *is* available
- Completes within 5 seconds for selections under 1,000 words

2.2.5 FR-AI-5 – Suggestion Review (MUST)

This is arguably more important than the AI generation itself. If users can't easily accept, reject, or tweak suggestions, the feature is useless.

Trigger: AI returns a suggestion.

Behavior: The suggestion is displayed alongside the original in a diff-style view.

Acceptance Criteria:

- Users can accept the suggestion (replaces original)
- Users can reject it (nothing changes)
- Users can *edit* the suggestion before accepting – partial acceptance
- Ctrl+Z / Cmd+Z undoes an accepted suggestion
- The outcome (accepted / rejected / modified) is logged for analytics (see FR-AI-6)

2.2.6 FR-AI-6 – AI Interaction Logging (SHOULD)

We need to track how the AI features are used, both for product analytics, admin auditing, and to let team leads understand how content evolved. But we have to be careful about what we log.

Trigger: Any AI request is processed.

Behavior: The system records metadata: operation type, timestamp, user ID, document ID, input / output text lengths, and whether the suggestion was accepted, rejected, or edited.

Acceptance Criteria:

- Full document content is *never* logged – only lengths and metadata
- Logs are kept for 30 days max, then purged automatically (aligned with NFR-SEC-5 retention policy)
- Workspace admins can see aggregated stats (not individual logs) through the admin dashboard
- Individual users can view their own AI interaction history for their documents (operation type, timestamp, outcome) – enabling team leads to review how a section evolved
- Logs are exportable for compliance audits (aggregated format, no content)

2.2.7 FR-AI-7 – AI Feature Policy and Quota Management (MUST)

Several stakeholders need control over who can use AI features and how much: workspace admins want per-role toggles, department managers and investors want cost controls, and the LLM provider's rate limits require per-user throttling.

Trigger: A workspace admin configures AI policies, or a user approaches / exceeds their usage quota.

Behavior: The system enforces admin-configured AI feature policies and tracks per-user and per-organization AI usage against configurable quotas.

Acceptance Criteria:

- Workspace admins can enable / disable specific AI operations (Rewrite, Summarize, Translate, Reformat) per role
- Per-user quotas are configurable (e.g., 100 AI invocations per day for free tier, 500 for paid)
- Per-organization budgets can be set as monthly limits on total AI invocations or estimated API cost
- Users see a warning when they reach 80% of their quota
- When the quota is exceeded, AI features are disabled with a clear message showing the reset time
- Policy changes take effect within 10 seconds for active sessions
- Cached / repeated operations (e.g., translating the same text to the same language) do not count against the quota

2.3 Document Management

Capability Statement: *The system shall support the full document lifecycle – creation from templates, automatic versioning with AI change tracking, granular sharing and permission control, structured commenting, folder-based organization, and export to common formats.*

2.3.1 FR-DOC-1 – Document Creation (MUST)

Trigger: User clicks "Create Document," optionally picking a template.

Behavior: A new document is created (blank or from template). The user who created it becomes the owner.

Acceptance Criteria:

- Document is ready to edit within 1 second
- Creator gets the "owner" role automatically
- Shows up in the user's document list right away
- At least 3 built-in templates available (blank, meeting notes, project brief)
- Workspace admins can create and manage custom templates for their organization

2.3.2 FR-DOC-2 – Version History (MUST)

Trigger: Changes are saved (auto or manual).

Behavior: The system snapshots the document periodically and on key events (like accepting an AI suggestion).

Acceptance Criteria:

- Users can browse a timeline of previous versions with timestamps and who made the changes
- Restoring an old version creates a *new* entry (non-destructive) – you can always go back
- History retained for at least 90 days
- AI-generated changes are tagged so you can tell them apart from manual edits

2.3.3 FR-DOC-3 – Sharing and Permissions (MUST)

Trigger: User clicks "Share" on a document.

Behavior: The system asks for a recipient (email or share link) and a permission level.

Acceptance Criteria:

- Four permission levels: Owner, Editor, Commenter, Viewer (matching the roles in FR-USR-2)
- Invitation sent within 5 seconds
- Share-by-link with configurable access (view-only or comment-only)
- Owners can revoke or change access for anyone
- Permission changes take effect within 10 seconds, even for users who are currently in the document

2.3.4 FR-DOC-4 – Exporting Documents (SHOULD)

Trigger: User picks a format from the export menu.

Behavior: The system converts the document and starts a download.

Acceptance Criteria:

- Supported: PDF, DOCX, Markdown
- Under 3 seconds for documents up to 100 pages
- Formatting, headings, and lists are preserved
- AI-generated content isn't flagged in exports by default, but there's a "track AI changes" toggle if you want it

2.3.5 FR-DOC-5 – Commenting (MUST)

We defined a "Commenter" role, so commenting needs to actually work as a first-class feature.

Trigger: User selects text and clicks "Add Comment."

Behavior: A comment is anchored to the selected range. Others can reply. Editors or Owners can resolve it.

Acceptance Criteria:

- Visible to everyone with document access
- Supports threaded replies
- Resolved comments move to a "Resolved" panel (not deleted)
- Viewers can read comments but not add them
- Comments sync across sessions within 1 second

2.3.6 FR-DOC-6 – Document Organization (SHOULD)

Without folders, the document list becomes a mess once you have more than a dozen documents.
Trigger: User creates a folder or moves a document.
Behavior: The document's location updates across all users who have access.
Acceptance Criteria:

- Users can create, rename, and delete folders
- Documents can be moved via drag-and-drop or a move dialog
- Admins can see the folder structure for their workspace
- Search works across all folders

2.4 User Management

Capability Statement: *The system shall provide secure authentication (including SSO for enterprise), role-based authorization with four distinct permission levels, secure session management, and user profile management to support collaboration identity.*

2.4.1 FR-USR-1 – Authentication (MUST)

Trigger: User tries to log in.
Behavior: Credentials are verified through the auth service.
Acceptance Criteria:

- Email / password with reasonable strength requirements (8+ chars, mixed case, one special character)
- OAuth 2.0 via Google and Microsoft at minimum
- SSO via SAML 2.0 for enterprise workspaces (required to meet Corporate Users' integration needs from Section 1.1)
- Account locks for 15 minutes after 5 failed attempts
- Successful login lands the user on their document dashboard within 2 seconds

2.4.2 FR-USR-2 – Role-Based Authorization (MUST)

Trigger: User tries to do anything on a document or workspace.
Behavior: The system checks whether the user's role allows the action.
Acceptance Criteria:

- Four roles with clearly defined capabilities:
  - **Owner** – full control: sharing, deletion, role assignment, version restore, all AI features
  - **Editor** – edit, invoke AI, comment, export
  - **Commenter** – view and comment only; no editing, no AI
  - **Viewer** – read-only; can't edit, comment, or use AI
- Unauthorized actions get a clear message ("You need Editor access to use this feature")
- Role changes propagate within 10 seconds to active sessions
- Workspace admins can toggle AI features per role via FR-AI-7

2.4.3 FR-USR-3 – Session Management (SHOULD)

Trigger: User logs in.
Behavior: A session token is created and maintained for the duration of activity.
Acceptance Criteria:

- Sessions expire after 30 minutes of inactivity
- Tokens stored securely (HTTP-only, Secure, SameSite cookies or similar)
- Logout invalidates the token on both client and server immediately
- Users can see and kill active sessions from account settings ("Log out of all devices")

2.4.4 FR-USR-4 – User Profile Management (COULD)

A lower-priority feature, but important for the collaboration UX – you need display names and avatars so people can tell each other apart.
Trigger: User goes to profile settings.
Behavior: They can update their display name, avatar, and notification preferences.
Acceptance Criteria:

- Name changes reflected in presence indicators and comments within 10 seconds
- Avatars limited to 2MB, PNG or JPEG
- Configurable notifications for share invitations and comment mentions

2.4.5 FR-USR-5 – Access Request Notifications (SHOULD)

When a viewer or other restricted user wants elevated access, they shouldn't have to go outside the product to ask for it.

Trigger: A user with insufficient permissions clicks "Request Edit Access" (or "Request Comment Access") on a document.

Behavior: The system sends a notification to the document owner with the requester's identity and the requested permission level. The owner can approve or deny directly from the notification.

Acceptance Criteria:

- Notification delivered to the owner within 10 seconds (in-app) and within 1 minute (email, if enabled)
- Request includes: requester display name, email, current role, requested role, and document name
- Duplicate requests from the same user for the same document within 24 hours are suppressed
- Owner can approve or deny from the notification without opening the document

3 Non-Functional Requirements

All the quality attributes that make the difference between a system that works in a demo and one that works in production. We tried to attach concrete, testable numbers to everything, and where possible we've included the reasoning behind the specific targets. For each NFR, we also describe how it would be verified.

3.1 System Performance

3.1.1 NFR-PERF-1 – Keystroke Propagation Latency (MUST)

When you type something in a shared document, the other users need to see it fast enough that it feels real-time.

Trigger: A user types in a shared document.
Behavior: The edit is synced to all active sessions.
Acceptance Criteria:

- Other users see the change within 500ms
- p99 latency stays under 1 second
- Cursor position updates within 300ms

Why 500ms? Research on groupware latency (Gutwin et al., 2004) shows that above roughly 500ms, users lose the sense that they're editing "together." They start to doubt whether their edits went through, leading to duplicate keystrokes and confusion. Below 100ms feels instant; 100–500ms feels responsive. We're targeting the upper bound of "responsive" since it's achievable over real-world networks.

Verification: Automated load testing with simulated concurrent editors using k6 or Artillery, measuring p50 / p95 / p99 latency under 10, 25, and 50 concurrent editor scenarios. Tests run on every release candidate against a staging environment that mirrors production topology.

3.1.2 NFR-PERF-2 – AI Response Initiation (MUST)

AI features involve a round trip to an external model, so they're inherently slower than a keystroke. But users still need to feel like something is happening.

Trigger: User invokes an AI action.
Behavior: Request goes to the AI service; response streams back.
Acceptance Criteria:

- Visible loading indicator within 300ms (so the user knows it's working)
- First tokens start appearing within 3 seconds
- Full response within 6 seconds for typical selections (around 500 words)

Why these numbers? Nielsen's response time research says users stay focused when feedback comes within 1 second, but for generative AI, people are conditioned to wait a bit longer as long as they see *something* happening (a spinner, a progress bar, the first few words streaming in). Beyond 6 seconds, attention drops off significantly. Streaming is key here – it makes a 5-second response feel much faster than waiting 5 seconds for a full block of text to appear.

Verification: Integration tests against the AI service (with both live and mock providers) measuring time-to-first-token and time-to-completion across 100 representative text samples of varying length (100, 500, 1000, 2000 words). Continuous monitoring in production via distributed tracing with latency percentile dashboards.

3.1.3 NFR-PERF-3 – Document Load Time (MUST)

Trigger: User opens a document.

Behavior: The system fetches document data and loads the editor, prioritizing the visible portion.

Acceptance Criteria:

- Documents under 1MB: within 2 seconds
- Documents up to 10MB: within 5 seconds
- The editor is interactive before background data (collaboration state, version history) finishes loading

Why these numbers? Google's research on page load times found that going from 1 to 3 seconds increases bounce probability by 32%. For a productivity app, 2 seconds for a normal-sized document keeps us competitive. Progressive rendering means you can start typing even if the full collaboration state hasn't loaded yet.

Verification: Synthetic monitoring with Lighthouse CI or WebPageTest measuring Time to Interactive (TTI) for documents at 100KB, 1MB, 5MB, and 10MB sizes. Regression tests on every deployment; monthly cross-browser testing (Chrome, Firefox, Safari, Edge).

3.2 Scalability

Expected Growth Model

We based our targets on the trajectory of comparable SaaS collaborative tools at similar stages (e.g., early-stage Notion and Coda):

- **Launch (months 1–3):** around 1,000 active users, around 5,000 documents, up to 10 concurrent editors on popular docs
- **Growth (months 4–12):** around 50,000 users, around 200,000 documents, up to 50 concurrent editors
- **Scale (year 2+):** 500K+ users, 2M+ documents, same latency and availability targets

Each phase should support at least 10x growth without requiring a major re-architecture.

Justification for Platform-Wide Targets

The concurrent-document and session targets below are derived from the growth model using peak-hour assumptions. At the Scale phase with 500K+ users, assuming 20% are active during peak hours (100K users), each with an average of 1.2 open documents, we get approximately 120K concurrently open documents. Rounding up to 100K+ as a minimum target provides headroom for organic growth before requiring re-architecture. The 1M active session target accounts for multiple tabs / devices per user and WebSocket connections for both editing and presence.

3.2.1 NFR-SCL-1 – Concurrent Editors per Document (MUST)

Trigger: Many users open the same document.
Behavior: The collaboration service keeps everyone in sync.
Acceptance Criteria:

- Up to 50 concurrent editors per document
- Latency targets from NFR-PERF-1 still hold at 50 editors
- Document consistency maintained through conflict resolution (FR-RTC-3)

Verification: Load tests simulating 50 concurrent editors typing at realistic rates (approximately 5 keystrokes / second each) while measuring p99 sync latency and verifying document convergence.

3.2.2 NFR-SCL-2 – Concurrent Documents (MUST)

Trigger: Large numbers of users working on different documents across the platform.
Behavior: Load is distributed across services.
Acceptance Criteria:

- 100,000+ documents open concurrently
- Up to 1,000,000 active sessions
- Per-document latency targets hold regardless of total platform load

Verification: Platform-wide load tests using distributed test harnesses (e.g., k6 with multiple injection points) simulating the Scale phase user distribution. Soak tests running for 4+ hours to detect memory leaks and connection pool exhaustion.

3.2.3 NFR-SCL-3 – Horizontal Scalability (SHOULD)

Trigger: Usage grows beyond current capacity.
Behavior: New service instances are spun up to handle the load.
Acceptance Criteria:

- Collaboration, API, and AI services scale independently of each other
- Supports 10x growth from the initial base (1K to 10K+) without re-architecture
- Auto-scaling policies based on CPU and active connections
- New instances operational within 2 minutes

Verification: Chaos engineering tests (e.g., Gremlin or LitmusChaos) that artificially spike load and verify auto-scaling triggers fire within the 2-minute window. Quarterly capacity planning reviews against actual growth metrics.

3.3 Availability

3.3.1 NFR-AVL-1 – Service Availability (MUST)

Trigger: Normal operations over time.
Behavior: The system stays up.
Acceptance Criteria:

- Core editing: at least 99.9% monthly (roughly 43 min downtime / month max)
- AI features: at least 99.5% (accounts for LLM provider dependency)
- Read-only access: at least 99.95%

Why tiered targets? Google Docs and Notion aim for roughly 99.95%. As a startup with fewer resources, 99.9% for core features is ambitious but realistic. The lower AI target (99.5%) reflects the reality that we depend on an external service we don't control. The tiered design ensures that even if the AI is down, people can still edit their documents.

Verification: External uptime monitoring (e.g., Pingdom or Better Uptime) with 1-minute check intervals hitting separate health endpoints for each tier: editing, AI, and read-only. Monthly SLA reports generated automatically.

3.3.2 NFR-AVL-2 – Editing Session Resilience (MUST)

Trigger: Network drops or a partial system failure.
Behavior: Client buffers edits locally and syncs when the connection comes back.
Acceptance Criteria:

- No confirmed edits lost
- Auto-retry on reconnection
- Document state is consistent post-reconnection (uses FR-RTC-3 conflict resolution)
- Local buffer survives browser refresh (IndexedDB or equivalent)

Verification: E2E tests that simulate network disconnection (using browser DevTools protocol or Toxiproxy), inject edits during offline window, restore connectivity, and verify zero data loss and state convergence.

3.3.3 NFR-AVL-3 – AI Service Failure Handling (SHOULD)

Trigger: AI provider goes down or rate limits kick in.
Behavior: Editing continues; user gets a clear error message about the AI issue.
Acceptance Criteria:

- Error notification within 2 seconds ("AI temporarily unavailable" or "usage limit reached")
- Document editing completely unaffectedw
- User can retry manually later
- Client-side error categorization: transient (retry soon), rate-limited (wait), down (check back later)

Verification: Integration tests with a mock AI provider that returns 503, 429, and timeout errors. Verify the editor remains fully functional and correct error categories are displayed. Chaos tests in staging that kill the AI service and confirm zero impact on editing latency.

3.3.4 NFR-AVL-4 – Disaster Recovery (MUST)

Trigger: Major failure – data center outage, DB corruption, etc.
Behavior: Restore from backups.
Acceptance Criteria:

- RTO: service restored within 4 hours
- RPO: at most 1 hour of data lost (hourly backups minimum)
- Daily automated backups to geographically separate storage
- DR procedures tested at least once per quarter

Verification: Quarterly DR drills with documented results: actual restore time measured against 4-hour RTO, data integrity checked against the 1-hour RPO, and drill report reviewed by the Maintenance Team.

3.4 Security and Privacy

Especially important given that we're sending user content to third-party AI services.

3.4.1 NFR-SEC-1 – Data Encryption in Transit (MUST)

Trigger: Any client-server communication.
Behavior: All traffic encrypted.
Acceptance Criteria:

- TLS 1.2 or higher everywhere
- WebSocket connections use WSS only
- Expired or self-signed certs are rejected

Verification: Automated SSL / TLS scanning (e.g., SSL Labs API or testssl.sh) on every deployment. CI pipeline rejects any configuration that allows TLS ¡ 1.2 or non-WSS WebSocket endpoints.

3.4.2 NFR-SEC-2 – Data Encryption at Rest (MUST)

Trigger: Document data is persisted.
Behavior: Encrypted storage.
Acceptance Criteria:

- AES-256 (or equivalent) for document data
- Keys managed through a KMS, not hardcoded anywhere
- Backups encrypted too

Verification: Infrastructure-as-code audit confirming encryption settings on all storage resources (database, object storage, backup buckets). Periodic penetration testing verifying that raw storage access does not expose plaintext document content.

3.4.3 NFR-SEC-3 – AI Processing Privacy (SHOULD)

This is the greatest privacy issue in the system. We're sending user-written content to a third-party API. We need to minimize what gets sent.

Trigger: User invokes AI on selected text.

Behavior: Only the selected text and a limited context window are sent.

Acceptance Criteria:

- Only selected text + up to 500 tokens of surrounding context included in the request
- No user IDs, document metadata, or unrelated content in the payload
- TLS for communication with the provider
- A data processing agreement with the LLM provider that prohibits using our requests for training
- Input sanitization applied to prevent prompt injection attacks – user-submitted text is wrapped in delimiters and the system prompt is isolated from user content to prevent the model from leaking system prompts or other users' data

Verification: Unit tests on the AI request builder confirming payload contains only selected text + context window and no extraneous metadata. Automated scanning of outbound AI payloads in staging for PII leakage. Annual third-party security audit of the AI integration pipeline, including prompt injection testing with adversarial inputs.

3.4.4 NFR-SEC-4 – AI Interaction Logging (SHOULD)

Trigger: AI request processed.

Behavior: Minimal metadata logged: type, timestamp, user ID, doc ID, text lengths, outcome.

Acceptance Criteria:

- No full document content or AI response text in the logs
- 30-day retention, then auto-purged
- Accessible to admins (for usage monitoring) and ops team (for debugging)

Verification: Log auditing script that scans AI interaction logs for any entry exceeding metadata-only fields (flag if any field is more than 200 characters, which would suggest content leakage). Automated test confirming logs older than 30 days are purged within 24 hours.

3.4.5 NFR-SEC-5 – Data Retention and Right to Erasure (SHOULD)

GDPR compliance means users have the right to be forgotten. We need to support that.

Trigger: User requests account deletion or a retention policy expires.

Behavior: All associated data is permanently deleted – documents, versions, comments, AI logs, everything.

Acceptance Criteria:

- Deletion completed within 30 days (GDPR requirement)
- Deleted data is unrecoverable

- Workspace admins can set retention periods per workspace
- Users can export their data before deletion (data portability, per GDPR Article 20)

Verification: Automated integration test that creates a user, populates data across all stores (documents, versions, comments, AI logs, profile), requests deletion, and verifies no traces remain in any data store after 30 days. Annual GDPR compliance audit.

3.4.6 NFR-SEC-6 – Prompt Injection Defense (SHOULD)

User documents may contain adversarial text designed to manipulate the LLM into ignoring its system instructions, leaking the system prompt, or producing harmful output. Since user content is directly embedded in AI requests, we need defense-in-depth.

Trigger: User-submitted text is included in an AI request.

Behavior: The system sanitizes and isolates user content before sending it to the LLM.

Acceptance Criteria:

- User-submitted text is enclosed in clear delimiters (e.g., XML tags) that separate it from system instructions
- The system prompt explicitly instructs the model to treat delimited content as data, not instructions
- AI responses are validated before display: responses that contain system prompt fragments or other users' data are rejected and a generic error is shown
- Known prompt injection patterns (e.g., "ignore previous instructions," "disregard the above," role-play directives) are flagged in monitoring and logged for security review, but not blocked at the input stage. This is a deliberate trade-off: user documents routinely contain text that resembles injection patterns without malicious intent – a legal contract might include "ignore previous instructions" as literal clause language, an AI research paper might quote injection techniques, or a cybersecurity training document might contain adversarial examples. Blocking at the input stage would produce false positives that silently break the AI feature for legitimate content, eroding user trust. Instead, defense is layered: input delimiters and system prompt isolation (criteria 1–2 above) prevent injections from taking effect, output validation (criterion 3) catches any that slip through, and monitoring flags let the security team identify new attack patterns and refine defenses iteratively without disrupting users.

Verification: Adversarial test suite with 50+ known prompt injection patterns run against the AI pipeline in CI. Quarterly red-team exercises targeting the AI integration.

3.5 Usability

3.5.1 NFR-USB-1 – Large Document Handling (SHOULD)

Trigger: User opens a big document (over 1MB, many pages).

Behavior: The visible section loads first; the rest fills in behind the scenes.

Acceptance Criteria:

- Above-the-fold content rendered and editable first
- Background sections load progressively

- Editing possible within 3 seconds even for 10MB documents

Verification: Performance tests with synthetic documents at 1MB, 5MB, and 10MB measuring time-to-interactive for the visible viewport. Regression tests on every release.

3.5.2 NFR-USB-2 – Collaboration UI at Scale (SHOULD)

With 30 people in a document, having 30 colored cursors bouncing around is chaos. The UI needs to handle this gracefully without degrading the editing experience.

Trigger: More than 10 collaborators in one document.

Behavior: The system limits visual clutter while maintaining awareness.

Acceptance Criteria:

- The collaboration panel loads within 1 second regardless of collaborator count
- No frame rate drops below 30fps with up to 50 concurrent editors
- Cursor rendering is throttled to avoid layout thrashing – updates batched at 100ms intervals when editor count exceeds 20
- The full collaborator list is accessible via an expandable panel

Verification: Automated rendering performance tests using Chrome DevTools Protocol to measure frame rates with 10, 25, and 50 simulated collaborators. Visual regression tests confirming the UI remains usable at each tier.

3.5.3 NFR-USB-3 – Accessibility (SHOULD)

Trigger: Users rely on assistive technologies (screen readers, keyboard navigation, switch devices).

Behavior: The interface is operable and understandable through assistive technologies across all core workflows.

Acceptance Criteria:

- WCAG 2.1 Level AA compliance across all stable UI surfaces (document list, settings, sharing dialogs, export, profile management).
- Keyboard navigation for all major actions: text formatting, AI invocation, commenting, sharing, and version history browsing.
- Color contrast: 4.5:1 minimum for normal text, 3:1 for large text.
- ARIA labels on all interactive elements (AI menu, collaboration indicators, comment threads, language picker).
- AI suggestions and their accept/reject/edit controls are fully keyboard-accessible and announced by screen readers. The diff preview (FR-AI-5) provides a text-based summary (e.g., "AI suggests replacing 3 sentences – Tab to review changes") in addition to the visual diff.
- Presence indicators (FR-RTC-2) are announced as live region updates when collaborators join or leave, but *not* on every cursor movement – continuous cursor announcements would make the editor unusable with a screen reader. Instead, a keyboard shortcut (e.g., Ctrl+Shift+P) reads out the current collaborator list and their approximate locations on demand.

Accessibility Risk Assessment: WCAG 2.1 AA compliance in a real-time collaborative editor is significantly harder than in a static web application. Three areas carry the highest accessibility risk:

1. **Live collaboration state (FR-RTC-2, FR-RTC-4):** Continuous cursor movements and change highlights from 20+ collaborators cannot be announced individually without overwhelming screen reader users. Our approach is to treat presence as an on-demand query rather than a live stream for assistive technology users, while keeping the visual experience unchanged for sighted users.

2. **AI suggestion diffs (FR-AI-5):** Inline diffs are inherently visual. For screen reader users, we provide a linearized text summary of changes (insertions, deletions, modifications) that can be navigated sequentially. This is a known hard problem – no major collaborative editor has solved it perfectly – so we target "usable" rather than "equivalent experience" for this specific interaction.

3. **Rich-text editing:** The editor itself (contentEditable or a custom input model) must expose document structure to the accessibility tree. We will prioritize compatibility with the two most widely used screen readers (NVDA on Windows, VoiceOver on macOS) and accept that edge cases in less common assistive technologies may require iterative fixes post-launch.

We prioritize accessibility compliance in the following order: (1) document reading and navigation, (2) commenting and sharing workflows, (3) AI suggestion review, (4) real-time collaboration presence. This ordering reflects both user impact (reading is the most common task) and implementation feasibility (static UI is easier to make accessible than dynamic collaboration state).

Verification: Automated accessibility scanning via axe-core integrated into the CI pipeline, catching regressions on every commit. Manual testing with NVDA (Windows) and VoiceOver (macOS) on every major release, covering the core flows: document editing, AI invocation and suggestion review, commenting, and sharing. A dedicated accessibility audit is conducted quarterly, with findings tracked as prioritized backlog items. We maintain an accessibility conformance report documenting known limitations and their planned remediation timeline.

3.5.4 NFR-USB-4 – Internationalization and RTL Support (SHOULD)

The translation feature (FR-AI-4) supports 20+ languages including Arabic, Russian, and other RTL scripts. The editor itself must handle these correctly.

Trigger: User creates or edits a document containing RTL text, mixed-direction content, or non-Latin scripts.

Behavior: The editor renders text direction, cursor movement, and selection correctly for all supported languages.

Acceptance Criteria:

- Full Unicode support in the editor, including CJK characters, Arabic / Chinese scripts, emoji, and combining characters
- Automatic text direction detection (LTR / RTL) at the paragraph level
- Correct cursor movement and text selection in RTL and bidirectional (mixed LTR+RTL) paragraphs
- AI-translated RTL output displays correctly in the suggestion preview (FR-AI-5)

- Collaboration features (cursors, comments, presence) work correctly in RTL documents

Verification: E2E tests with documents in Arabic, Hebrew, Chinese, Japanese, and mixed-direction content. Manual QA with native RTL speakers on each major release.

3.6 Data Consistency

3.6.1 NFR-CON-1 – Document State Convergence (MUST)

The system uses eventual consistency for collaborative editing. We need a bounded staleness guarantee so that users are never looking at arbitrarily stale document states.

Trigger: Multiple users edit the same document concurrently.

Behavior: All clients converge to an identical document state after edits propagate.

Acceptance Criteria:

- All collaborators observe the same document state within 2 seconds of the last edit under normal network conditions (bounded staleness)
- After a network partition heals, convergence occurs within 5 seconds (aligned with FR-RTC-5)
- No operation reordering that changes semantic meaning – the CRDT / OT algorithm preserves user intent
- The server-side document state is the authoritative source of truth; client states that diverge beyond the staleness bound trigger a forced resync

Why bounded staleness? In a collaborative editor, users need to be confident that what they see is close to what others see. Unbounded eventual consistency creates situations where two users believe they're editing together but are actually working on divergent states, leading to surprise merges. The 2-second bound matches our latency targets (NFR-PERF-1) and ensures that the collaboration experience feels coherent.

Verification: Convergence tests that inject concurrent edits from multiple simulated clients, introduce artificial network delays (up to 2 seconds), and verify all clients reach identical document state within the specified bounds. Fuzz testing with randomized edit sequences to detect edge cases in the CRDT / OT implementation.

4 User Stories and Scenarios

We wrote fourteen user stories spanning all four required areas: collaboration, AI assistant workflows, document lifecycle, and access control. Each story follows the standard format ("As a . . . , I want . . . , so that . . . ") and includes acceptance criteria that tie back to the functional and non-functional requirements from Sections 2 and 3. Where the expected system behavior isn't obvious, we explain the design rationale.

4.1 Collaboration Scenarios

4.1.1 US-1: Simultaneous Paragraph Editing

As a document editor working on a shared report,
I want to edit the same paragraph that a colleague is editing at the same time,
so that we can both work freely without coordinating who types where.

Scenario: Alice is rewriting the introduction while Bob inserts a sentence into the same paragraph. Both are typing at the same time.

Expected Behavior:

- Both users' keystrokes appear in real time (within 500ms per NFR-PERF-1).
- The CRDT/OT algorithm (FR-RTC-3) merges the edits automatically – no "conflict resolution" dialog, no lost text.
- Each user's cursor is visible in a distinct color (FR-RTC-2), so Alice can see where Bob is typing and adjust accordingly.
- After both stop typing, all clients converge to the same document state within 2 seconds (NFR-CON-1).

Design Rationale: We chose invisible, automatic conflict resolution over a manual merge dialog (like Git) because collaborative editing happens at keystroke granularity. Asking users to resolve conflicts every few seconds would make the product unusable. The trade-off is that the merge result may occasionally surprise a user (e.g., two insertions at the same position end up in an order neither expected), but this is a well-understood trade-off accepted by every major collaborative editor.

Related Requirements: FR-RTC-1, FR-RTC-3, FR-RTC-2, NFR-PERF-1, NFR-CON-1

4.1.2 US-2: Offline Editing and Reconnection

As a user editing a document on a train with spotty connectivity,
I want to keep typing even when my internet drops and have my changes sync automatically when I'm back online,
so that I don't lose my work or have to remember where I left off.

Scenario: Charlie is editing a proposal. His connection drops for 10 minutes. During that time, he continues writing two new paragraphs. Meanwhile, Dana (who is online) edits the document's conclusion.

Expected Behavior:

- Charlie's edits are cached locally in IndexedDB (NFR-AVL-2). The editor remains fully functional.
- A visual indicator shows Charlie that he is offline (FR-RTC-5).
- On reconnection, cached edits sync within 5 seconds (FR-RTC-5). The same conflict resolution algorithm (FR-RTC-3) handles merging Charlie's and Dana's changes.
- No edits from either user are lost. The local buffer survives even if Charlie accidentally refreshes the browser while offline (NFR-AVL-2).

Related Requirements: FR-RTC-5, FR-RTC-3, NFR-AVL-2, NFR-CON-1

4.1.3 US-3: Reverting to a Previous Version During Active Collaboration

As a document owner,
I want to revert to a previous version of the document while other people are still editing,
so that I can undo a large unwanted change without waiting for everyone to stop working.

Scenario: Eve discovers that an AI-assisted rewrite from an hour ago introduced factual errors across several sections. Frank and Grace are currently editing other parts of the document.

Expected Behavior:

- Eve opens the version history timeline (FR-DOC-2), identifies the version from before the AI rewrite (AI-generated changes are tagged per FR-DOC-2), and clicks "Restore."
- Restoring creates a *new* version – it does not overwrite history. Eve (or anyone) can undo the revert if it turns out to be wrong.
- Frank and Grace see their editors update to the restored state. Any edits they made after Eve clicked "Restore" are merged on top of the restored version using the standard conflict resolution algorithm (FR-RTC-3).
- A notification or change indicator (FR-RTC-4) highlights the affected regions so Frank and Grace understand what happened.

Design Rationale: Version restore is treated as a large edit, not a global reset, so it flows through the same sync pipeline as regular keystrokes. This avoids a special "lock the document" mode and keeps the experience consistent. The risk is that Frank's or Grace's in-flight edits may conflict with the restored content, but the CRDT/OT algorithm is designed to handle exactly this case.

Related Requirements: FR-DOC-2, FR-RTC-3, FR-RTC-4, NFR-CON-1

4.1.4 US-4: Presence Awareness in a Crowded Document

As a team member in a 25-person sprint retrospective document,
I want to see who else is in the document and where they are typing without the UI becoming overwhelming,
so that I can find an empty section to contribute to and avoid editing on top of someone.

Expected Behavior:

- Collaborators appear in a sidebar list with their name and color (FR-RTC-2).
- With 25 editors, cursor rendering is throttled – updates are batched at 100ms intervals to prevent layout thrashing (NFR-USB-2).
- The frame rate stays above 30fps (NFR-USB-2).
- Only the nearest few cursors are shown inline by default; the full list is accessible via an expandable panel (NFR-USB-2).

Related Requirements: FR-RTC-2, NFR-USB-2, NFR-SCL-1

4.2 AI Assistant Workflows

4.2.1 US-5: Summarizing a Long Section

As a user reviewing a 1,500-word meeting transcript pasted into a document,
I want to select the transcript and ask the AI to summarize it,
so that I can insert a concise summary at the top of the document for people who didn't attend.

Scenario: The user selects the full transcript, clicks the AI button (or presses the keyboard shortcut), and picks "Summarize."

Expected Behavior:

- The AI action menu appears within 300ms (FR-AI-1).
- A loading indicator appears immediately. The summary begins streaming within 3 seconds and completes within 5 seconds for this text length (FR-AI-3, NFR-PERF-2).
- The summary is at most 30% the length of the original – roughly 450 words (FR-AI-3).
- The summary appears in a preview panel. The user can insert it at their cursor position or discard it (FR-AI-3, FR-AI-5).
- Only the selected text plus up to 500 tokens of surrounding context are sent to the LLM provider – no user IDs, no document metadata (NFR-SEC-3).
- The interaction is logged as metadata only: operation type ("Summarize"), input length, output length, timestamp, outcome (FR-AI-6, NFR-SEC-4).

Related Requirements: FR-AI-1, FR-AI-3, FR-AI-5, FR-AI-6, NFR-PERF-2, NFR-SEC-3, NFR-SEC-4

4.2.2 US-6: Translating a Section into Another Language

As a user preparing a bilingual document,
I want to select a paragraph in English and translate it into French,
so that I can include both versions in my report without leaving the editor.

Scenario: The user selects a 200-word paragraph, picks "Translate," and chooses French from the language picker.

Expected Behavior:

- A language picker appears showing all 20+ supported languages (FR-AI-4).
- The translated text is shown in a preview alongside the original (FR-AI-5). The user can compare, edit the translation, and then insert it.
- If the user translates the same paragraph to French again later, the cached result is returned without consuming an AI quota invocation (FR-AI-7).
- The translation completes within 5 seconds for this text length (FR-AI-4).
- If the text were in Arabic, the preview correctly renders RTL direction (NFR-USB-4).

Related Requirements: FR-AI-4, FR-AI-5, FR-AI-7, NFR-PERF-2, NFR-USB-4

4.2.3 US-7: Partially Accepting an AI Rewrite

As a user who asked the AI to rewrite a paragraph for conciseness,
I want to accept some of the AI's changes and reject others,
so that I keep control over the final wording rather than having to choose all-or-nothing.

Scenario: The user selects a paragraph and picks "Rewrite" with a "more concise" prompt. The AI returns a version that rewrites three sentences. The user likes two of the rewrites but prefers the original phrasing of the third.

Expected Behavior:

- The suggestion appears as an inline diff or side panel (FR-AI-5), with the original text visible for comparison.

- The user can directly *edit* the AI suggestion before accepting – they delete the third rewritten sentence and paste back the original (FR-AI-5).
- Clicking "Accept" replaces the original text with the user's modified version.
- A version history entry is created, tagged as an AI-generated change (FR-DOC-2). The original text is recoverable via Ctrl+Z / Cmd+Z (FR-AI-5).
- The outcome is logged as "modified" (not "accepted" or "rejected") for analytics (FR-AI-6).

Design Rationale: Many AI writing tools only offer accept/reject. We allow editing the suggestion before committing because users frequently want the AI's direction but not its exact words. This makes the AI feel like a collaborator rather than an autocomplete.

Related Requirements: FR-AI-2, FR-AI-5, FR-AI-6, FR-DOC-2

4.2.4 US-8: AI Invocation During Active Collaboration

As a user collaborating in real time with colleagues,
I want to invoke the AI assistant on a paragraph while others are editing elsewhere in the document,
so that AI assistance doesn't block or disrupt anyone's work.

Scenario: Alice selects a paragraph and requests an AI rewrite. While the AI is processing (up to 6 seconds), Bob is editing a different section, and Carol starts typing in the same paragraph Alice selected.

Expected Behavior:

- Alice sees a loading indicator on her selected text. Bob is unaffected – his edits propagate normally.
- Carol's edits to the same paragraph are *not* blocked. When Alice's AI response arrives, it is based on the text as it was when Alice initiated the request. The suggestion preview shows a diff against that original snapshot, not against the current (now-modified) state.
- If Alice accepts the suggestion, it is applied as an edit that flows through the normal conflict resolution pipeline (FR-RTC-3). Carol's concurrent edits are merged with the AI-produced text the same way any two overlapping edits would be.
- A presence indicator or subtle annotation shows other users that Alice has an AI operation pending on that region – similar to how cursors show where someone is working.

Design Rationale: We chose not to lock the paragraph during AI processing. Locking would be simpler to implement (no concurrent edit merging needed), but it blocks collaborators for up to 6 seconds, which is unacceptable in a real-time editor. The trade-off is that the AI's output may be stale by the time Alice reviews it – but since Alice previews before committing (FR-AI-5), she can see if the suggestion still makes sense.

Related Requirements: FR-AI-2, FR-AI-5, FR-RTC-3, NFR-PERF-2, NFR-CON-1

4.2.5 US-9: Hitting the AI Usage Quota

As a free-tier user who has been using the AI assistant heavily today,
I want to know when I'm approaching my limit and what happens when I hit it,
so that I can plan my remaining AI usage or decide to upgrade.

Scenario: The user has made 80 out of 100 daily AI invocations. They invoke the AI again.
Expected Behavior:

- At 80 invocations (80% of quota), a non-intrusive warning appears: "You've used 80 of 100 AI requests today. Resets at midnight UTC." (FR-AI-7).
- The user can continue using AI until they hit 100.
- At 100 invocations, the AI button shows "Daily limit reached – resets at midnight UTC" (FR-AI-7). Clicking it does nothing except display this message.
- All non-AI features (editing, collaboration, commenting, sharing) remain fully functional.
- If the user repeats a previous request (e.g., translating the same text to the same language), the cached result is returned without counting against the quota (FR-AI-7).

Related Requirements: FR-AI-7, FR-AI-1, NFR-AVL-3

4.2.6 US-10: Restructuring Notes into a Document Outline

As a user with a disorganized set of bullet points and notes pasted into a document,
I want to select them and ask the AI to restructure them into a coherent outline with headings and subheadings,
so that I can quickly turn raw meeting notes or brainstorming output into a well-organized draft without doing it manually.

Scenario: A product manager has pasted 40 bullet points from a brainstorming session into a document. The points cover three loosely related themes but are in no particular order. She selects the entire block, clicks the AI button, and picks "Reformat."

Expected Behavior:

- The AI action menu appears within 300ms (FR-AI-1). The user selects "Reformat" and is prompted with restructuring options (e.g., "Organize into outline," "Group by theme," "Sort chronologically").
- Only the selected text plus up to 500 tokens of surrounding context is sent to the LLM provider – no user IDs, no document metadata, no unrelated content (NFR-SEC-3).
- Because the selection is under 2,000 words, the restructured output begins streaming within 3 seconds and completes within 6 seconds (NFR-PERF-2, FR-AI-2).
- The result appears in a diff-style preview (FR-AI-5), showing the original flat list on one side and the proposed structured outline on the other. The user can see that the AI has grouped the 40 bullet points under four thematic headings and reordered them.
- The user notices that two bullet points were placed under the wrong heading. She edits the suggestion directly in the preview panel, moving them to the correct section, before clicking "Accept" (FR-AI-5 – partial acceptance).
- Accepting the restructured version creates a version history entry tagged as an AI-generated change (FR-DOC-2). The original flat list is recoverable via Ctrl+Z / Cmd+Z.
- The interaction is logged as metadata: operation type ("Reformat"), input length, output length, timestamp, outcome ("modified") (FR-AI-6, NFR-SEC-4).

Design Rationale: Restructuring is qualitatively different from rewriting or summarizing – it changes the organization of content rather than its wording or length. This makes the diff preview especially important: users need to verify that the AI has grouped ideas correctly, not just that the prose reads well. We present restructuring options ("Organize into outline," "Group by theme") rather than a single generic reformat because the intent behind restructuring varies – a chronological sort is very different from a thematic grouping, and the AI cannot reliably infer the user's intent without guidance. The option to edit before accepting is critical here, since even a good restructuring will often place a few items in the wrong category.

Related Requirements: FR-AI-1, FR-AI-2, FR-AI-5, FR-AI-6, FR-DOC-2, NFR-PERF-2, NFR-SEC-3, NFR-SEC-4

Related Requirements: FR-USR-2, FR-RTC-2, FR-DOC-3

4.3 Document Lifecycle

4.3.1 US-11: Sharing a Document with Read-Only Access

As a document owner,
I want to share my document with a client via a link that only allows viewing,
so that the client can read the document without accidentally (or intentionally) changing anything.

Scenario: The owner clicks "Share," generates a view-only link, and sends it to the client.

Expected Behavior:

- The share dialog offers a link with configurable access: view-only or comment-only (FR-DOC-3).
- The client opens the link and sees the document in read-only mode. The editing toolbar is hidden. AI features are unavailable (FR-USR-2: Viewer role has no AI access).
- The client *can* see other users' cursors (FR-RTC-2) if live editing is happening – this builds trust that the document is current.
- The owner can revoke the link at any time; access is removed within 10 seconds even if the client has the document open (FR-DOC-3).

Related Requirements: FR-DOC-3, FR-USR-2, FR-RTC-2

4.3.2 US-12: Exporting with AI Change Tracking

As a team lead preparing a document for external review,
I want to export the document as a PDF with AI-generated changes visually distinguished,
so that reviewers know which parts were written by humans and which were AI-assisted.

Scenario: The document contains several paragraphs that were rewritten using the AI assistant over the past week. The team lead needs to send a PDF to an external compliance reviewer who requires transparency about AI involvement. She also plans to share a DOCX version with a co-author who uses Word's review features.

Expected Behavior:

- In the export dialog (FR-DOC-4), there is a "Track AI changes" toggle. By default, it is off – exports look clean.
- When enabled, AI-generated or AI-modified text is annotated in the export. The annotation style depends on the format:

- **PDF:** AI-assisted sections are highlighted with a subtle background color and a margin note reading "AI-assisted" with the operation type (e.g., "AI Rewrite," "AI Translation").
- **DOCX:** AI changes are represented as tracked changes (using Word's native revision markup), so the reviewer can accept or reject them individually in Word – bridging our AI workflow with a familiar review paradigm.
- **Markdown:** AI-assisted sections are wrapped in HTML comment annotations (e.g., `<!-- AI: Rewrite -->`) that are invisible when rendered but preserved in the source for auditability.

- This is possible because the version history tags AI-generated changes with their operation type (FR-DOC-2), and the AI interaction log records which text ranges were affected (FR-AI-6). The export pipeline combines these two data sources to determine annotation ranges.
- The export completes within 3 seconds for a document under 100 pages (FR-DOC-4).
- If a section was AI-generated and then manually edited by a user, the annotation reflects this: "AI-assisted (modified)" rather than plain "AI-assisted," so reviewers can distinguish between unreviewed AI output and human-curated AI output.

Design Rationale: We chose format-specific annotation strategies rather than a single generic approach because each export format has different affordances. PDF is a final-output format, so annotations should be visual but non-intrusive. DOCX is a working format, so leveraging Word's native tracked-changes model lets reviewers interact with AI changes using tools they already know – they can accept or reject individual AI rewrites without returning to our platform. Markdown is a developer-oriented format where invisible metadata is more appropriate than visual markup. The distinction between "AI-assisted" and "AI-assisted (modified)" addresses a real compliance concern: regulators and reviewers care not just whether AI was involved, but whether a human reviewed and refined the AI's output.

Related Requirements: FR-DOC-4, FR-DOC-2, FR-AI-6

4.3.3 US-13: Reviewing AI Interaction History

As a team lead,
I want to review the AI interaction history for a specific document to understand how a particular section evolved,
so that I can assess whether the team is using AI appropriately and whether the AI's contributions improved the document.

Scenario: The team lead opens a document that five team members have been working on. During review, she notices that an entire 800-word section reads unusually uniformly – the style is consistent in a way that suggests a single AI rewrite rather than incremental human editing. She wants to investigate how this section was produced.

Expected Behavior:

- The team lead opens the AI interaction history panel (FR-AI-6) for the document.
- The panel shows a chronological list: timestamp, user, operation type (Rewrite / Summarize / Translate / Reformat), input and output text lengths, and outcome (accepted / rejected / modified).

- She filters by the relevant date range and sees that a team member invoked "Rewrite" on the section and accepted the result without modification (outcome: "accepted," input length: 820 words, output length: 790 words).
- Full document content is not shown in the logs – only metadata (FR-AI-6, NFR-SEC-4). To see the actual text before and after the rewrite, the team lead navigates to the version history (FR-DOC-2), where the AI-generated change is tagged. She can compare the pre-rewrite and post-rewrite versions side by side.
- Combined, the AI interaction log and version history let the team lead reconstruct the editorial process: who asked the AI to do what, when, and what the document looked like before and after.
- Based on this review, she leaves a comment on the section (FR-DOC-5) asking the team member to manually review the AI output for factual accuracy, since the rewrite was accepted wholesale without modification.

Design Rationale: Separating the AI interaction log (metadata only) from the version history (full content) is a deliberate privacy-preserving design. The interaction log can be retained, aggregated, and shown to workspace admins without exposing document content – satisfying the observability needs of team leads and compliance officers (NFR-SEC-4). When a team lead needs to see actual content changes, they use the version history, which is subject to the document's own access controls. This two-layer approach means we can offer useful AI auditing without creating a secondary store of sensitive document content in the logs. The scenario above illustrates a real governance concern: if a team member accepts AI output without review, the team lead should be able to detect and address that pattern.

Related Requirements: FR-AI-6, FR-DOC-2, FR-DOC-5, NFR-SEC-4

4.4 Access Control and Roles

4.4.1 US-14: Commenter Attempts to Use the AI Assistant

As a user with the "Commenter" role on a shared document,
I want to understand why the AI assistant is unavailable to me,
so that I know it's a permissions issue and not a bug.

Scenario: A commenter selects text in the document and looks for the AI button.

Expected Behavior:

- The AI button is visible but grayed out (not hidden – hiding it would confuse users who have seen it in other documents).
- Hovering over the grayed-out button shows a tooltip: "AI features require Editor access. Contact the document owner to change your role." (FR-USR-2, FR-AI-1).
- The commenter can still add comments (FR-DOC-5) and read the document.
- If the owner upgrades the commenter to Editor, the AI button activates within 10 seconds without a page refresh (FR-USR-2, FR-AI-7).

Design Rationale: We show the button (grayed out with a tooltip) rather than hiding it because hiding features based on role creates an inconsistent UI – users wonder if the feature exists or if something is broken. A visible-but-disabled state with a clear explanation is more transparent.

Related Requirements: FR-USR-2, FR-AI-1, FR-AI-7, FR-DOC-5

4.4.2 US-15: Workspace Admin Configures AI Policies per Role

As a workspace administrator at a legal firm,
I want to disable the AI "Rewrite" feature for associates while keeping "Summarize" and "Translate" enabled,
so that associates don't accidentally replace sensitive contract language with AI-generated text, but can still use AI for productivity tasks that don't alter the document.

Scenario: The admin opens the workspace settings and navigates to the AI policy panel.
Expected Behavior:

- The admin sees a matrix of AI operations (Rewrite, Summarize, Translate, Reformat) × roles (Owner, Editor, Commenter, Viewer) with toggles for each combination (FR-AI-7).
- The admin disables "Rewrite" for the "Editor" role. "Summarize" and "Translate" remain enabled.
- Policy changes take effect within 10 seconds for all active sessions (FR-AI-7). Associates currently in a document see the "Rewrite" option disappear (or gray out) without refreshing.
- The admin can also set per-organization quotas: e.g., a monthly budget of 10,000 AI invocations across all users (FR-AI-7).
- All AI policy changes are recorded in the audit log (FR-AI-6).

Related Requirements: FR-AI-7, FR-USR-2, FR-AI-6

4.4.3 US-16: Viewer Attempts to Edit

As a user with "Viewer" access to a document shared via a read-only link,
I want the system to prevent me from editing and clearly explain why,
so that I understand my access level rather than thinking the editor is broken.

Scenario: A viewer clicks into the document body and tries to type.
Expected Behavior:

- The document content is not editable – there is no cursor insertion. Clicking into the text does not activate a text input.
- A banner at the top of the editor reads: "You have view-only access. Contact the owner for editing permissions." (FR-USR-2).
- The viewer cannot invoke AI features, add comments, or export (per the Viewer role definition in FR-USR-2).
- The viewer *can* see real-time edits from other collaborators (FR-RTC-2) and browse the document freely.
- If the viewer needs to request access, a "Request Edit Access" button sends a notification to the document owner.

5 Requirements Traceability

This section provides the traceability matrix linking every user story (Section 4) to its supporting functional and non-functional requirements (Sections 2–3) and to the architecture components that implement them (Part 2). The matrix serves two purposes: (1) confirming that every user story is backed by concrete requirements and realized by identifiable components, and (2) confirming that no requirement is orphaned – every requirement traces to at least one user story and at least one component.

5.1 Requirement Validation Mapping

The following abbreviations refer to the major containers and components from the C4 architecture (Part 2). Each component's responsibility is summarized briefly here for traceability purposes.

ID	Component	Responsibility
FE	Frontend Editor	Rich-text editor UI, presence display, AI suggestion panels, offline local buffer (IndexedDB), diff previews
RTS	Real-Time Sync Service	WebSocket gateway, CRDT/OT engine, cursor broadcast, reconnection handling
AIS	AI Assistant Service	LLM API abstraction, prompt construction, response caching, quota enforcement, interaction logging
DS	Document Service	Document CRUD, version history, template management, export pipeline, folder organization
AUS	Auth & User Service	Authentication (email/OAuth/SAML SSO), role-based authorization, session management, profile storage
AG	API Gateway	Request routing, rate limiting, TLS termination, input validation
AD	Admin Dashboard	Workspace management, AI policy configuration, usage analytics, audit log export
NS	Notification Service	Email invitations, in-app alerts (share invites, comment mentions, quota warnings)

Table 3: Architecture component abbreviations

5.2 User Story Traceability Matrix

Table 4 maps each user story to the functional requirements, non-functional requirements, and architecture components that support it.

Table 4: Traceability matrix: user stories → requirements → components

User Story	Functional Req.	Non-Functional Req.	Component
US-1: Simultaneous Paragraph Editing	FR-RTC-1, FR-RTC-2, FR-RTC-3	NFR-PERF-1, NFR-CON-1	FE, RTS
US-2: Offline Editing and Reconnection	FR-RTC-3, FR-RTC-5	NFR-AVL-2, NFR-CON-1	FE, RTS
US-3: Reverting During Active Collaboration	FR-DOC-2, FR-RTC-3, FR-RTC-4	NFR-CON-1	FE, RTS, DS
US-4: Presence in Crowded Document	FR-RTC-2	NFR-USB-2, NFR-SCL-1	FE, RTS
US-5: Summarizing a Long Section	FR-AI-1, FR-AI-3, FR-AI-5, FR-AI-6	NFR-PERF-2, NFR-SEC-3, NFR-SEC-4	FE, AIS, AG
US-6: Translating a Section	FR-AI-4, FR-AI-5, FR-AI-7	NFR-PERF-2, NFR-USB-4	FE, AIS, AG
US-7: Partially Accepting AI Rewrite	FR-AI-2, FR-AI-5, FR-AI-6, FR-DOC-2	–	FE, AIS, DS
US-8: AI During Active Collaboration	FR-AI-2, FR-AI-5, FR-RTC-3	NFR-PERF-2, NFR-CON-1	FE, AIS, RTS
US-9: Hitting AI Usage Quota	FR-AI-1, FR-AI-7	NFR-AVL-3	FE, AIS, AG
US-10: Restructuring Notes into a Document Outline	FR-AI-1, FR-AI-2, FR-AI-5, FR-AI-6, FR-DOC-2	NFR-PERF-2, NFR-SEC-3, NFR-SEC-4	FE, AIS, DS
US-11: Sharing with Read-Only Access	FR-DOC-3, FR-USR-2, FR-RTC-2	–	FE, DS, AUS, NS
US-12: Export with AI Change Tracking	FR-DOC-2, FR-DOC-4, FR-AI-6	–	FE, DS, AIS
US-13: Reviewing AI Interaction History	FR-AI-6, FR-DOC-2	NFR-SEC-4	FE, AIS, DS

Continued on next page

User Story	Functional Req.	Non-Functional Req.	Component
US-14: Commenter Attempts AI	FR-USR-2, FR-AI-1, FR-AI-7, FR-DOC-5	–	FE, AUS, AIS
US-15: Workspace Admin Configures AI Policies	FR-AI-6, FR-AI-7, FR-USR-2	–	AD, AUS, AIS
US-16: Viewer Attempts to Edit	FR-USR-2, FR-RTC-2, FR-DOC-3, FR-USR-5	–	FE, AUS, DS, NS

5.3 Requirement Coverage Matrix

Table 5 confirms the reverse mapping: every functional requirement is traced to at least one user story and at least one architecture component. This ensures no requirement is orphaned.

Table 5: Reverse traceability: functional requirements → user stories → components

Requirement	Traced from User Stories	Components
Real-Time Collaboration		
FR-RTC-1	US-1	FE, RTS
FR-RTC-2	US-1, US-4, US-11, US-16	FE, RTS
FR-RTC-3	US-1, US-2, US-3, US-8	FE, RTS
FR-RTC-4	US-3	FE, RTS
FR-RTC-5	US-2	FE, RTS
AI Writing Assistant		
FR-AI-1	US-5, US-9, US-14	FE, AIS
FR-AI-2	US-7, US-8	FE, AIS
FR-AI-3	US-5	FE, AIS
FR-AI-4	US-6	FE, AIS
FR-AI-5	US-5, US-6, US-7, US-8	FE, AIS
FR-AI-6	US-5, US-7, US-12, US-13, US-15	AIS, DS, AD
FR-AI-7	US-6, US-9, US-14, US-15	AIS, AG, AD
Document Management		

Continued on next page

Requirement	Traced from User Stories	Components
FR-DOC-1	–[a]	FE, DS
FR-DOC-2	US-3, US-7, US-12, US-13	DS, FE
FR-DOC-3	US-11, US-16	DS, AUS, NS
FR-DOC-4	US-12	DS, FE
FR-DOC-5	US-14	FE, DS, RTS
FR-DOC-6	–[a]	FE, DS
User Management		
FR-USR-1	–[a]	AUS, AG
FR-USR-2	US-11, US-14, US-15, US-16	AUS, FE, AD
FR-USR-3	–[a]	AUS
FR-USR-4	–[a]	AUS, FE
FR-USR-5	US-16	NS, AUS, FE

[a] *Note:* FR-DOC-1 (Document Creation), FR-DOC-6 (Document Organization), FR-USR-1 (Authentication), FR-USR-3 (Session Management), and FR-USR-4 (User Profile) are foundational requirements exercised implicitly by nearly every user story (e.g., all stories presuppose an authenticated user working within a document). They do not trace to a single dedicated user story because they are preconditions for all stories rather than the focus of any particular scenario. Their inclusion in the requirements specification is justified by the stakeholder analysis (Section 1), and their implementation is covered by the architecture components listed above.

5.4 Non-Functional Requirement Coverage

For completeness, Table 6 shows how non-functional requirements map to user stories and components. NFRs are cross-cutting by nature, so many apply broadly rather than to a single story.

Table 6: Non-functional requirement coverage

NFR	Exercised by User Stories	Components
Performance		
NFR-PERF-1	US-1	FE, RTS
NFR-PERF-2	US-5, US-6, US-8	AIS, AG
NFR-PERF-3	Cross-cutting	FE, DS, AG
Scalability		
NFR-SCL-1	US-4	RTS
NFR-SCL-2	Cross-cutting	RTS, DS, AG
NFR-SCL-3	Cross-cutting	All services

Continued on next page

NFR	Exercised by User Stories	Components
Availability		
NFR-AVL-1	Cross-cutting	All services
NFR-AVL-2	US-2	FE, RTS
NFR-AVL-3	US-9	AIS, FE
NFR-AVL-4	Cross-cutting	DS, RTS
Security & Privacy		
NFR-SEC-1	Cross-cutting	AG, RTS
NFR-SEC-2	Cross-cutting	DS
NFR-SEC-3	US-5	AIS
NFR-SEC-4	US-5, US-13	AIS
NFR-SEC-5	Cross-cutting	DS, AUS, AIS
NFR-SEC-6	Cross-cutting	AIS
Usability		
NFR-USB-1	Cross-cutting	FE
NFR-USB-2	US-4	FE, RTS
NFR-USB-3	Cross-cutting	FE
NFR-USB-4	US-6	FE
Data Consistency		
NFR-CON-1	US-1, US-2, US-3, US-8	RTS, FE

5.5 Traceability Observations

Several patterns emerge from the matrices above that are worth noting:

High-coupling components. The Frontend Editor (FE) and Real-Time Sync Service (RTS) appear in the largest number of traces. This is expected: the editor is the user-facing surface for almost every feature, and real-time sync underpins all collaboration scenarios. This concentration of traces confirms that these two components carry the most architectural risk and should receive the most testing attention.

AI Assistant as a cross-cutting concern. The AI Assistant Service (AIS) traces to seven of the fifteen user stories, spanning collaboration (US-8), document lifecycle (US-12, US-13), access control (US-14, US-15), and dedicated AI workflows (US-5, US-6, US-7, US-9). This validates the architectural decision to treat AI integration as a first-class service with well-defined interfaces rather than a bolted-on feature.

Foundational requirements without dedicated stories. Five functional requirements (FR-DOC-1, FR-DOC-6, FR-USR-1, FR-USR-3, FR-USR-4) do not map to a specific user story. As noted above, these are infrastructure-level capabilities that enable all other stories. Rather than writing artificial user stories for "a user logs in," we acknowledge these as preconditions. Their testability is ensured through the acceptance criteria defined in Section 2, and their architectural coverage is confirmed by the component mapping.

Cross-cutting NFRs. Most non-functional requirements labeled "cross-cutting" (e.g., NFR-AVL-1, NFR-SEC-1, NFR-SCL-3) are verified through system-wide testing strategies (load tests,

security scans, chaos engineering) rather than through individual user stories. Their verification methods are detailed in Section 3.

Part 2: System Architecture

Contents

1 Part 2: System Architecture (45%) 2

1.1 Architectural Drivers . . . 2

1.1.1 How driver ranking influenced the architecture . . . 2

1.2 System Design using the C4 Model . . . 3

1.2.1 Level 1 — System Context Diagram . . . 3

1.2.2 Level 2 — Container Diagram . . . 4

1.2.3 Level 3 — Component Diagram . . . 5

1.3 Feature Decomposition . . . 5

1.4 AI Integration Design . . . 6

1.4.1 Context and scope . . . 6

1.4.2 Suggestion UX . . . 6

1.4.3 AI during collaboration . . . 6

1.4.4 Prompt design . . . 6

1.4.5 Model and cost strategy . . . 6

1.5 API Design . . . 7

1.5.1 Document CRUD . . . 7

1.5.2 Real-time session management . . . 7

1.5.3 AI assistant invocation . . . 7

1.5.4 User and permission management . . . 7

1.5.5 API style choices . . . 7

1.5.6 Long-running AI operations . . . 8

1.5.7 Error handling strategy . . . 8

1.6 Authentication & Authorization . . . 8

1.7 Communication Model . . . 8

1.8 Code Structure & Repository Organization . . . 9

1.9 Data Model . . . 9

1.10 Architecture Decision Records (ADRs) . . . 11

1 Part 2: System Architecture (45%)

1.1 Architectural Drivers

Before presenting the architecture, the main architectural drivers are identified and ranked according to how strongly they influence the design.

1. **Real-time collaboration** This is the most important driver because the system is centered around multiple users editing the same document simultaneously. It directly affects the choice of communication model, synchronization strategy, and conflict handling.

2. **Low latency and responsiveness** Users must see edits and AI feedback quickly. This influences the use of push-based communication, optimistic UI updates, and asynchronous AI execution.

3. **Scalability** The system should support many concurrent users, documents, and AI requests. This drives the separation into multiple containers and services.

4. **Security and access control** Since documents may contain private information, the architecture must enforce authentication, authorization, and controlled sharing.

5. **AI integration as a core feature** The AI assistant is not an external add-on. It affects document workflows, API design, versioning, and collaboration behavior.

6. **Maintainability and modularity** The system should be decomposed into modules that can be developed and tested independently, which influences service boundaries and repository organization.

1.1.1 How driver ranking influenced the architecture

The ranking of architectural drivers directly shaped the final design choices.

- **Real-time collaboration** was ranked first, so the architecture uses a dedicated real-time synchronization service and push-based communication rather than periodic polling. This decision prioritizes consistency and immediate propagation of edits between collaborators.

- **Low latency and responsiveness** was ranked second, which led to lightweight frontend interactions, asynchronous AI execution, and separation between standard CRUD APIs and real-time update channels. This ensures users do not wait for long-running AI tasks before continuing to edit.

- **Scalability** was ranked third, which justified decomposition into multiple containers such as the API gateway, real-time service, document-related backend services, and AI assistant service. This separation allows different parts of the system to scale independently.

- **Security and access control** influenced the inclusion of authentication and authorization as explicit architectural concerns rather than implementation details. This is reflected in the authentication service, identity provider integration, permission model, and controlled document sharing.

- **AI integration as a core feature** led to the decision to model AI as a dedicated service with its own components, request lifecycle, and quota management instead of embedding it inside general backend logic.
- **Maintainability and modularity** influenced the monorepo structure, the separation into shared packages, and the use of ADRs to document major design choices clearly.

This means the architecture is not only a decomposition of functionality, but also a direct response to the ranked non-functional and product-level requirements. Different rankings would lead to different architectures. For example, if simplicity were ranked above collaboration, a polling-based design might be chosen instead of a real-time synchronization service.

1.2 System Design using the C4 Model

1.2.1 Level 1 — System Context Diagram

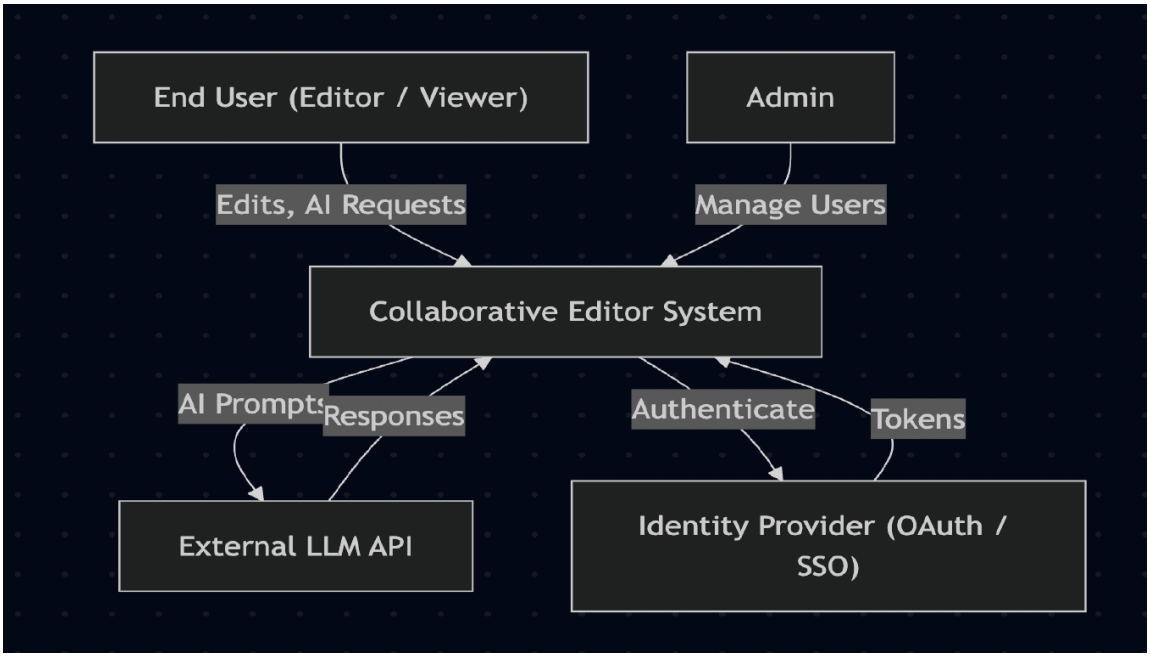

Figure 1: System Context Diagram

Explanation. The system interacts with two primary actor types: end users and administrators. End users create, edit, and review documents and invoke AI features. Administrators manage users and system-level access. The system also interacts with an external LLM API for AI-powered suggestions and an identity provider for authentication and token issuance.

1.2.2 Level 2 — Container Diagram

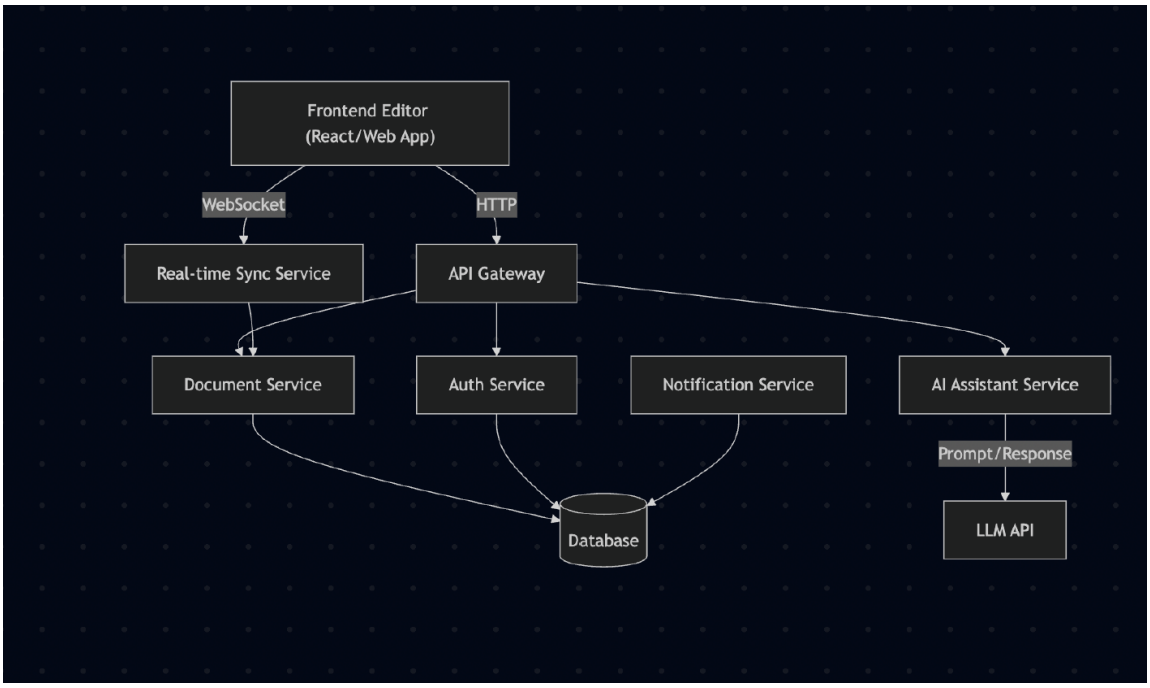

Figure 2: Container Diagram

Explanation. The frontend editor provides the user interface for editing and collaboration. It communicates with the API gateway over HTTP for standard application requests and with the real-time synchronization service for live updates. The backend is decomposed into document, authentication, notification, and AI assistant services. Persistent data is stored in the database, while the AI assistant service communicates with the external LLM API.

1.2.3 Level 3 — Component Diagram

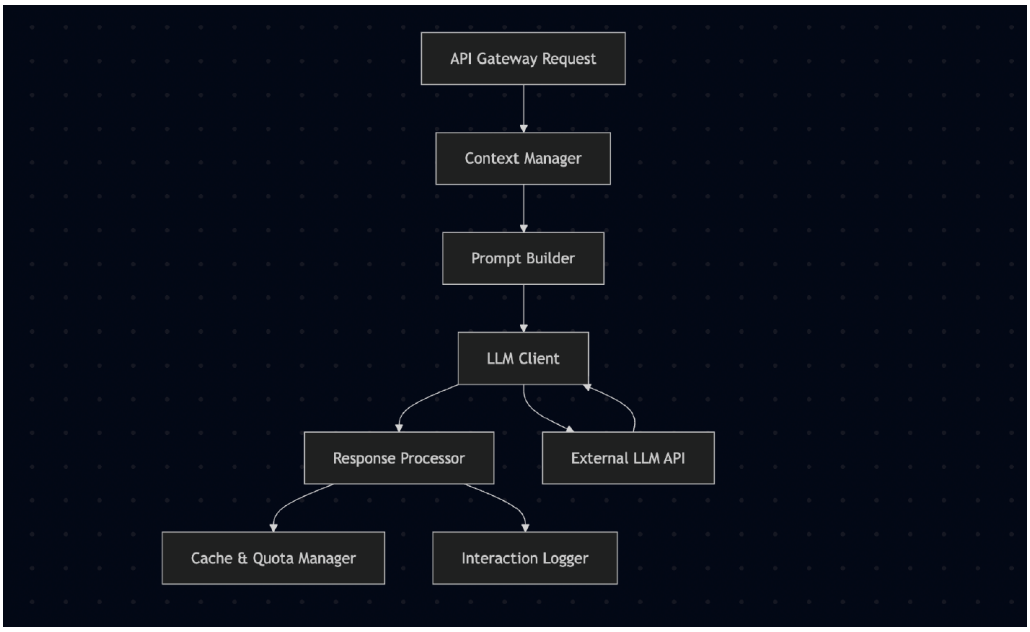

Figure 3: Component Diagram of the AI Assistant Service

Explanation. Inside the AI assistant service, the context manager gathers the relevant document context for the request. The prompt builder then constructs a feature-specific prompt. The LLM client sends the request to the external LLM API and receives the result. The response processor formats the returned output into suggestions that the editor can display. The cache and quota manager controls repeated requests and usage limits, while the interaction logger records AI operations for analytics and auditing.

1.3 Feature Decomposition

The architecture is decomposed into the following major modules.

- **Rich-text editor and frontend state management** Responsible for rendering the document, handling user input, formatting, cursor state, and local editor state. It depends on the API layer and real-time synchronization layer. It exposes editing actions and state updates to the rest of the frontend.
- **Real-time synchronization layer** Responsible for propagating edits between collaborators, maintaining document session state, and handling reconnection. It depends on the frontend editor, document service, and communication infrastructure. It exposes live change streams and session update events.
- **AI assistant service** Responsible for processing rewrite, summarize, translate, and restructure requests. It depends on prompt templates, context extraction, quota management, and the external LLM API. It exposes AI invocation endpoints and suggestion results.
- **Document storage and versioning** Responsible for storing document content, metadata, snapshots, and version history. It depends on the database and backend services. It exposes CRUD operations, version retrieval, and revert functionality.

- **User authentication and authorization** Responsible for identifying users and enforcing permissions. It depends on the identity provider and user management data. It exposes authenticated sessions and access checks.
- **API layer** Responsible for connecting the frontend to backend services through structured interfaces. It depends on the application services and shared validation logic. It exposes document, user, permission, AI, and collaboration endpoints.

1.4 AI Integration Design

1.4.1 Context and scope

By default, the AI assistant operates on the currently selected text because this provides the best balance between relevance, speed, and cost. For features such as summarization or restructuring, the system may include the enclosing section or, if needed, the entire document. For very long documents, the context manager truncates or chunks the content so that only the most relevant parts are included.

1.4.2 Suggestion UX

AI suggestions are presented as non-destructive proposals rather than immediate replacements. A side panel and inline suggestion view are both supported. Users can accept, reject, or partially apply a suggestion. Accepted suggestions remain reversible through the normal undo stack and version history.

1.4.3 AI during collaboration

If a user invokes AI on a region that is actively being edited, the system marks that region as pending AI generation. Other collaborators can still view it, but conflicting edits are either temporarily restricted or reconciled after the AI response returns, depending on the interaction type. This avoids silent overwrites while keeping the collaboration experience understandable.

1.4.4 Prompt design

Prompt construction is template-based rather than hardcoded directly into business logic. Each AI feature has a structured template with placeholders for document context, user instruction, and output constraints. This allows prompts to evolve independently of the rest of the system and supports future experimentation without major redeployment.

1.4.5 Model and cost strategy

Different AI tasks may use different models. Lightweight requests such as short rewrites can use a cheaper model, while more complex restructuring tasks can use a stronger model. Cost is controlled through per-user quotas and possibly organization-level limits. When a limit is exceeded, the system returns a clear quota-related error and disables further AI calls until the quota resets or is increased.

1.5 API Design

The API layer combines request-response interactions for standard operations with real-time communication for collaborative editing.

1.5.1 Document CRUD

- `POST /documents` — create a document
- `GET /documents/{documentId}` — fetch a document
- `PUT /documents/{documentId}` — update document metadata or content
- `DELETE /documents/{documentId}` — delete a document
- `GET /documents/{documentId}/versions` — list version history
- `POST /documents/{documentId}/revert` — revert to a previous version

1.5.2 Real-time session management

- `POST /sessions/join` — join a collaboration session
- `POST /sessions/leave` — leave a collaboration session
- `/realtime/document/{documentId}` — live synchronization channel

1.5.3 AI assistant invocation

- `POST /ai/rewrite`
- `POST /ai/summarize`
- `POST /ai/translate`
- `POST /ai/restructure`
- `GET /ai/jobs/{jobId}` — get AI job status

1.5.4 User and permission management

- `GET /users/{userId}`
- `POST /documents/{documentId}/share`
- `PUT /documents/{documentId}/permissions`
- `DELETE /documents/{documentId}/permissions/{principalId}`

1.5.5 API style choices

REST-style endpoints are used for CRUD, user management, and permission management because these are resource-oriented interactions. Real-time communication is used for collaborative editing because polling would create poor user experience and unnecessary latency. AI operations are exposed as asynchronous requests because they may take longer than standard CRUD operations.

1.5.6 Long-running AI operations

When the client invokes an AI feature, the backend creates a job and returns a job identifier immediately. The frontend then either polls for completion or subscribes to status events through the real-time layer. The result is displayed only when the job is marked complete.

1.5.7 Error handling strategy

Clients distinguish different AI states through explicit response codes and status values. For example:

- `processing` — the AI request is still running
- `failed` — the AI request failed internally or externally
- `quota_exceeded` — the user exceeded the allowed AI usage

1.6 Authentication & Authorization

Authentication is required because documents may be private, shared selectively, or editable only by specific users. The expected user types include regular end users, document owners, collaborators, viewers, and administrators.

The system supports the following roles:

- **Owner**: full control over the document, including sharing, editing, deleting, version reverts, and AI access management
- **Editor**: can modify content and invoke AI features, but cannot change ownership
- **Viewer**: can read the document but cannot edit it
- **Admin**: can manage users and system-wide access policies

Privacy is especially important because document content may be sent to third-party LLM APIs. The architecture should therefore minimize unnecessary context sharing, log AI requests carefully, and clearly define which content leaves the system boundary.

1.7 Communication Model

The system uses a real-time push-based communication model for collaborative editing. This gives users immediate visibility of remote edits, which is essential for a document editor. Compared with polling, this approach provides much better responsiveness, though it introduces more implementation complexity.

When a user opens a shared document, the client first fetches the latest persisted document state and then joins the active collaboration session. If the user loses connectivity, local changes are buffered if possible, and the client attempts to reconnect. Once reconnected, the client resynchronizes with the latest document state and resumes live updates.

1.8 Code Structure & Repository Organization

A monorepo is preferred because the team is relatively small and the frontend, backend, and shared contracts evolve together. This approach simplifies shared type definitions, validation logic, and coordinated changes.

```
root/
  apps/
    frontend/
    api-gateway/
    realtime-service/
    ai-service/
  packages/
    shared-types/
    shared-validation/
    prompt-templates/
  infrastructure/
  docs/
```

Top-level responsibilities:

- `apps/frontend`: editor UI and frontend state management
- `apps/api-gateway`: external API entry point
- `apps/realtime-service`: collaboration and synchronization logic
- `apps/ai-service`: prompt construction and LLM integration
- `packages/shared-types`: shared interfaces and DTOs
- `packages/shared-validation`: reusable validation rules
- `packages/prompt-templates`: AI prompt definitions

Secrets such as API keys, database URLs, and LLM credentials should be stored in environment variables or secret managers and never committed to the repository.

Tests should be organized close to the source code or in dedicated test directories. The testing strategy includes unit tests, integration tests, and end-to-end tests. AI integration can be tested using mocked provider responses rather than calling the real LLM every time.

1.9 Data Model

The data model captures the core entities required for collaboration, access control, versioning, and AI-assisted editing. Figure 4 shows the entity-relationship structure of the system.

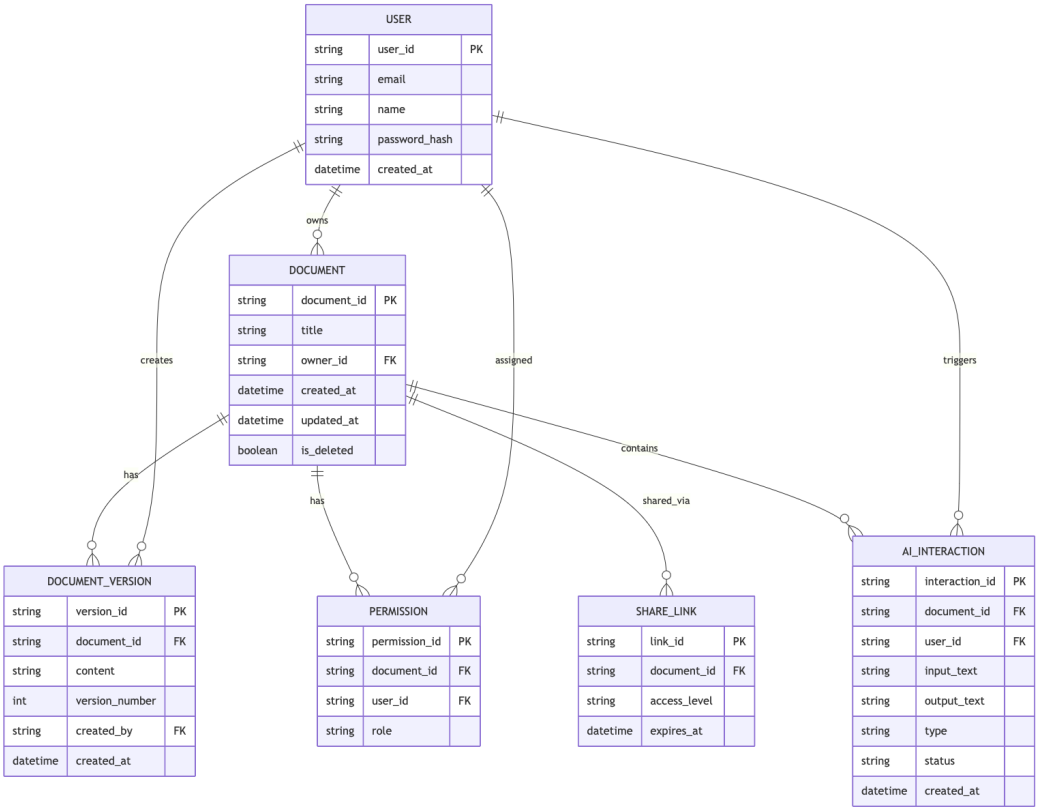

Figure 4: Entity-Relationship Diagram

Explanation. The ER diagram represents the persistent data layer of the system and the relationships between key entities.

- **USER** stores account information such as user ID, email, name, password hash, and creation timestamp. A user can own documents, be assigned permissions, create document versions, and trigger AI interactions.
- **DOCUMENT** is the central entity representing editable content. Each document has an owner and includes metadata such as title, timestamps, and deletion status. A document can be shared with other users and can have multiple versions.
- **DOCUMENT_VERSION** stores historical versions of a document. Each version is linked to a specific document and records the content snapshot, version number, creator, and timestamp. This enables version history, rollback, and auditing.
- **PERMISSION** defines access control between users and documents. It links a user to a document with a specific role such as viewer or editor, enabling secure and fine-grained collaboration.
- **SHARE_LINK** enables link-based sharing. Each share link is associated with a document and includes an access level and expiration time, allowing temporary or public access without explicitly assigning a user.

- **AI_INTERACTION** records AI-related operations. It links a user and a document, and stores the input text, generated output, request type, status, and timestamp. This supports monitoring, analytics, and quota enforcement.

The relationships reflect the system's functionality: a user owns documents, documents contain multiple versions, users can be granted permissions, documents can be shared via links, and users can trigger AI interactions on documents.

Relation to architectural drivers. The data model reflects the architectural drivers identified earlier. Real-time collaboration is supported through version tracking and shared access mechanisms. Security requirements are addressed through explicit ownership and role-based permissions. AI integration is treated as a core feature by modeling AI interactions as a first-class entity, enabling traceability, cost control, and auditing.

1.10 Architecture Decision Records (ADRs)

ADR 1: Use push-based real-time collaboration

Status: Accepted

Context: The product requires multiple users to edit the same document with immediate visibility of changes.

Decision: Use a real-time push-based synchronization model instead of polling.

Consequences: This improves collaboration quality and responsiveness, but increases implementation complexity.

Alternatives considered: Polling was rejected because it introduces delay and provides a weaker collaborative experience.

ADR 2: Separate AI assistant into its own service

Status: Accepted

Context: AI requests have different performance, cost, and scaling characteristics than core CRUD operations.

Decision: Isolate AI functionality into a dedicated service.

Consequences: This improves modularity and scalability, but adds service-to-service communication overhead.

Alternatives considered: Embedding AI logic directly inside the main backend was rejected because it would tightly couple unrelated concerns.

ADR 3: Use a monorepo for code organization

Status: Accepted

Context: Frontend and backend share contracts, validation rules, and coordinated feature changes.

Decision: Use a monorepo containing multiple apps and shared packages.

Consequences: This simplifies code sharing and development workflow, but requires discipline to keep boundaries clear.

Alternatives considered: A multi-repo structure was rejected because it would add friction for a relatively small team with tightly related components.

ADR 4: Use asynchronous execution for AI operations

Status: Accepted

Context: AI operations may take significantly longer than normal application requests.

Decision: Handle AI requests as asynchronous jobs with explicit status tracking.

Consequences: This avoids blocking the UI and improves reliability, but requires job tracking and more complex client handling.

Alternatives considered: Synchronous execution was rejected because it would create poor user experience for long-running requests.

Part 3 – Project Management & Team Collaboration
Collaborative Document Editor with AI Writing Assistant

Alexander Danchev, Anel Murat, Yintong Wang

February 2026

Contents

1 Team Structure & Ownership **2**
1.1 Cross-cutting Features . . . 2
1.2 Decision-Making Process . . . 2

2 Development Workflow **2**
2.1 Branching Strategy . . . 2
2.2 Merge Policy . . . 2
2.3 Code Review . . . 3
2.4 Issue Tracking and Task Assignment . . . 3
2.5 Communication . . . 3

3 Development Methodology **3**
3.1 Iteration Structure . . . 3
3.2 Backlog Prioritization . . . 4
3.3 Handling Non-User-Facing Work . . . 4

4 Risk Assessment **5**

5 Timeline and Milestones **5**
5.1 M1: Foundation Setup . . . 5
5.2 M2: Core Document Flow (CRUD + Versioning) . . . 5
5.3 M3: Authentication and Authorization . . . 6
5.4 M4: Real-Time Collaboration . . . 6
5.5 M5: AI Assistant Baseline . . . 7
5.6 M6: AI and Collaboration Integration . . . 7
5.7 M7: Reliability, Security, and Final Integration . . . 7

1 Team Structure & Ownership

Area	Owner	Responsibilities
Frontend & Collaboration	Alexander Danchev	Rich-text editor UI, frontend state management, presence indicators, and real-time user experience
AI Assistant	Anel Murat	AI service, prompt construction, quota management, and interaction logging
Backend & Core Services	Yintong Wang	API layer, document service, authentication, database, and overall system integration

Although the proof-of-concept (PoC) may initially be implemented primarily by a single team member to accelerate integration, ownership remains distributed. This ensures that each member can extend and maintain their respective components in later stages without creating bottlenecks.

1.1 Cross-cutting Features

Features such as AI-assisted editing during real-time collaboration span multiple subsystems. These are handled through:

- Clearly defined API contracts
- Shared data structures and type definitions
- Early agreement on interaction patterns

1.2 Decision-Making Process

Technical decisions are made collaboratively and justified based on requirements and architectural drivers. When disagreements arise, decisions are guided by the priorities defined in Part 2.

2 Development Workflow

2.1 Branching Strategy

The project follows a feature-branch workflow:

- The `main` branch is always stable and deployable
- Feature branches follow the naming convention:
  - `feature/<feature-name>`
  - `fix/<issue-name>`

2.2 Merge Policy

- All changes are integrated through pull requests
- At least one team member must review each pull request
- Direct commits to `main` are not allowed

2.3 Code Review

Code reviews focus on:

- Correctness
- Completeness
- Consistency with architecture and API contracts
- Readability and maintainability

2.4 Issue Tracking and Task Assignment

Tasks are managed using an issue tracking system (e.g., GitHub Issues). Each task includes:

- A clear description
- Acceptance criteria
- Assigned owner

Task states:

- To Do
- In Progress
- Done

2.5 Communication

The team uses Teams for daily communication. Important decisions are documented in:

- Architecture documentation (via GoogleDoc)
- Architecture Decision Records (via GoogleDoc)
- Pull request discussions

3 Development Methodology

The team follows a lightweight Agile methodology tailored to a small team and the exploratory nature of the project.

3.1 Iteration Structure

Development is organized into weekly iterations to enable rapid feedback and manageable progress. Each iteration consists of three phases:

- **Planning:**
  - Select tasks from the backlog based on priority and dependencies
  - Break down features into small, implementable units
  - Define clear acceptance criteria aligned with functional and non-functional requirements
- **Implementation:**

  - Develop features within assigned ownership areas (frontend, AI service, backend)
  - Follow predefined API contracts and shared data models
  - Integrate changes incrementally to reduce merge conflicts
  - Write tests alongside implementation where applicable

- **Review:**
  - Validate completed work against acceptance criteria
  - Perform code reviews and integration testing
  - Identify defects and inconsistencies early

3.2 Backlog Prioritization

The backlog is prioritized based on architectural dependencies and system risk. Features that form the foundation of the system or carry high technical uncertainty are implemented first. The prioritization order is as follows:

1. Core document functionality (creation, loading, persistence)
2. Real-time collaboration (synchronization and presence)
3. Authentication and authorization (security and access control)
4. AI assistant features (rewrite, summarize, translation)
5. Secondary features (comments, export, admin tools)

Tasks are also prioritized within each iteration to balance:

- Dependency constraints (what must be built first)
- Risk reduction (implementing uncertain components early)
- User value (delivering meaningful functionality incrementally)

3.3 Handling Non-User-Facing Work

Work that does not directly produce user-visible features is treated as a first-class part of the backlog rather than secondary or optional work. This includes infrastructure setup, data modeling, and testing.
Each such task is defined with:

- Clear deliverables (e.g., database schema, CI pipeline, API validation layer)
- Measurable acceptance criteria (e.g., passing tests, successful deployment)

Examples include:

- Setting up the monorepo structure and shared type definitions
- Designing the document and versioning data model
- Implementing automated testing and continuous integration

These tasks are scheduled early in the project because they directly affect the stability and scalability of later development. By explicitly tracking and prioritizing this work, the team avoids technical debt and ensures that core infrastructure supports all subsequent features.

4 Risk Assessment

Risk	Likelihood	Impact	Mitigation	Contingency
Real-time synchronization complexity	High	High	Use CRDT/OT approaches and incremental testing	Reduce advanced collaboration features
AI latency and cost variability	High	High	Use quotas, caching, and asynchronous processing	Disable or limit AI features temporarily
LLM API failure	Medium	Medium	Graceful degradation and error handling	Retry or fallback behavior
Data consistency issues	Medium	High	Convergence testing and validation	Force document resynchronization
Team coordination issues	Medium	Medium	Clear ownership and API contracts	Reassign responsibilities and simplify interfaces

5 Timeline and Milestones

The timeline extends beyond the current assignment and outlines the development plan for the remainder of the semester.

5.1 M1: Foundation Setup

Description: Establish the production-ready development environment and repository structure based on the architecture defined in Part 2.
Scope:

- Monorepo structure finalized (frontend, API gateway, real-time service, AI service)
- Shared types and validation packages implemented
- Environment configuration and secrets management set up
- Basic CI pipeline (build + lint + tests) configured

Acceptance Criteria:

- Repository can be cloned and run locally following README instructions
- All services start successfully in a local environment
- Shared type definitions are used across frontend and backend
- CI pipeline passes on the main branch

—

5.2 M2: Core Document Flow (CRUD + Versioning)

Description: Implement document lifecycle functionality end-to-end.
Scope:

- Document creation, retrieval, update, and deletion

- Version history and snapshot creation
- Basic frontend integration for document editing (single-user)

Acceptance Criteria:

- User can create, load, edit, and save a document via the frontend
- Version history is stored and retrievable
- Reverting to a previous version creates a new version entry
- Integration tests verify CRUD operations and version behavior

—

5.3 M3: Authentication and Authorization

Description: Implement user authentication and role-based access control.
Scope:

- Login and session management
- Role-based permissions (Owner, Editor, Commenter, Viewer)
- Document sharing and permission updates

Acceptance Criteria:

- Users can log in and maintain authenticated sessions
- Role restrictions are enforced (e.g., Viewer cannot edit, Commenter cannot use AI)
- Sharing a document updates permissions within 10 seconds
- Unauthorized actions are blocked with clear user feedback

—

5.4 M4: Real-Time Collaboration

Description: Enable multi-user editing with real-time synchronization.
Scope:

- Real-time session management
- Presence awareness (active users, cursors)
- Conflict resolution using CRDT/OT
- Offline editing and reconnection support

Acceptance Criteria:

- Multiple users can edit the same document simultaneously
- Edits propagate within latency targets (as defined in NFR-PERF-1)
- No data loss during concurrent editing or reconnection
- All clients converge to the same document state within defined bounds

—

5.5 M5: AI Assistant Baseline

Description: Implement core AI functionality with asynchronous execution.
Scope:

- AI endpoints (rewrite, summarize, translate, restructure)
- Prompt templates and context construction
- Asynchronous job handling and status tracking
- Quota management and basic caching

Acceptance Criteria:

- AI requests return a job ID and complete asynchronously
- Results are displayed in preview mode (non-destructive)
- Quota limits are enforced with appropriate user feedback
- Mock or real AI provider responses integrate correctly with the frontend

—

5.6 M6: AI and Collaboration Integration

Description: Integrate AI features into the collaborative editing workflow.
Scope:

- AI invocation during active collaboration
- Suggestion review (accept, reject, partial edit)
- Version tagging for AI-generated changes
- Interaction logging (metadata only)

Acceptance Criteria:

- AI suggestions can be applied without disrupting other collaborators
- Concurrent edits and AI outputs are merged consistently
- Accepted AI changes appear in version history with proper tagging
- AI interaction logs contain metadata only (no raw document content)

—

5.7 M7: Reliability, Security, and Final Integration

Description: Final system hardening, testing, and preparation for submission.
Scope:

- Error handling and graceful degradation (especially for AI failures)
- Logging and monitoring
- Security checks (encryption, access control validation)

- End-to-end testing of major workflows
- Demo preparation and documentation

Acceptance Criteria:

- All major user flows (editing, collaboration, AI usage) work end-to-end
- System behaves correctly under failure scenarios (e.g., AI unavailable)
- No sensitive document content is logged in AI interaction logs
- Final demo runs successfully with minimal setup

Part 4 – Proof of Concept
Collaborative Document Editor with AI Writing Assistant

Alexander Danchev, Anel Murat, Yintong Wang

February 2026

1 Proof of Concept Repository

The source code for the proof-of-concept implementation is available at the following Git repository:

https://github.com/ronesync/ai1220